

Contrastive-Learning-Based Decision Making for Dynamic Time-Linkage Optimization

Xiao-Fang Liu¹, Senior Member, IEEE, Meng Gao, Yongchun Fang², Senior Member, IEEE, Zhi-Hui Zhan³, Fellow, IEEE, and Jun Zhang⁴, Fellow, IEEE

Abstract—In dynamic time-linkage optimization, current decisions influence the future state of environments. To make good decisions that have a positive impact on future states, existing methods usually build a model to predict the future rewards of solutions for decision making. However, these prediction models present low accuracy since decision data are not enough to train such a complex model. To address this issue, this article proposes a contrastive-learning-based decision making (CLDM) method, which builds a contrastive model to learn the relationship between solutions but not absolute rewards and adopts a quick decision strategy to select solutions. In CLDM, a clustering-based time-linkage detection (CD) strategy is developed to measure the intensity of the time linkage, which determines whether to make decisions based on future rewards. To represent the relative relationship between solutions, a large number of contrastive samples are constructed using the limited historical decisions. A contrastive model is trained for solution comparison in terms of the combination of current fitness and future rewards. Candidate solutions are clustered into multiple groups to filter poor ones, and a few solutions are preserved to rank using the contrastive model. The winner is taken as the decision solution. Integrating CLDM into particle swarm optimization (PSO), a new algorithm named contrastive-learning-based PSO (CL-PSO) is put forward. Experimental results on multiple dynamic time-linkage optimization instances demonstrate that CL-PSO outperforms state-of-the-art algorithms in terms of solution quality. CL-PSO can also well solve the mobile robot path planning problem.

Index Terms—Contrastive learning, dynamic time-linkage optimization, evolutionary computation, particle swarm optimization (PSO), prediction.

Received 24 May 2025; accepted 15 September 2025. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62476141 and Grant 62233011, in part by the National Research Foundation of Korea (NRF) through Korea Government (MSIT) under Grant RS-2025-00555463 and Grant RS-2025-25456394, in part by Tianjin Top Scientist Studio Project under Grant 24JRRRCRC00030, and in part by Tianjin Belt and Road Joint Laboratory under Grant 24PTLYHZ00250. This article was recommended by Associate Editor S. Bechikh. (Corresponding authors: Zhi-Hui Zhan; Jun Zhang.)

Xiao-Fang Liu, Meng Gao, and Yongchun Fang are with the College of Artificial Intelligence, Nankai University, Tianjin 300350, China, and also with the Institute of Intelligence Technology and Robotic Systems, Shenzhen Research Institute of Nankai University, Shenzhen 518083, China (e-mail: liuxiaofang@nankai.edu.cn).

Zhi-Hui Zhan is with the College of Artificial Intelligence, Nankai University, Tianjin 300350, China (e-mail: zhanapollo@163.com).

Jun Zhang is with Nankai University, Tianjin 300350, China, also with Hanyang University, ERICA, Ansan 15588, South Korea, also with Zhejiang Normal University, Jinhua 321017, China, and also with Chaoyang University of Technology, Taichung 413310, Taiwan (e-mail: junzhang@iee.org).

This article has supplementary material provided by the authors and color versions of one or more figures available at <https://doi.org/10.1109/TSMC.2025.3611797>.

Digital Object Identifier 10.1109/TSMC.2025.3611797

I. INTRODUCTION

DYNAMIC time-linkage optimization problems widely exist in real-world applications [1], in which the environmental factors are influenced by historical decisions and change with time [2]. For example, in air combat, the battlefield situation greatly depends on the historical decisions of agents, resulting in the change in optimal decisions [3]. The dependence of environmental states on historical decisions brings great challenges to evolutionary algorithms [4].

Multiple methods are developed to track the change of optimal solutions but ignore the influence on future environments. They can be roughly categorized into three types [5], i.e., diversity enhancement (see [6]), prediction (see [7]), and transfer learning method (see [8]). Diversity enhancement methods introduce new solutions once a change occurs or maintain diversity along the evolutionary procedure [9]. Prediction methods assume that the change in optima follows stable patterns, which can be learned from historical data using machine learning techniques [10]. These methods work well on stable changing patterns but may fail on random changes. Transfer learning methods adopt data from historical and current environments to build a transfer model for predicting high-quality solutions [11]. These methods perform well on random changes. However, all these approaches make decisions based on current rewards only, resulting in poor performance in the long term [12], [13].

To deal with the time linkage of decisions, research efforts are paid to developing prediction models for evaluating the future rewards of solutions. For example, future rewards are predicted based on historical information using the least-squares method [14]. Since prediction models usually have errors, a stochastic-ranking selection scheme is proposed to make decisions through probability selection to reduce the impact of prediction errors [15]. Nonelitism is also adopted to improve algorithm performance [16]. In addition, the dynamic time-linkage problem is modeled as a multiple-decision-making problem, which can be solved using reinforcement learning [17]. To evaluate the long-term rewards of solutions, a surrogate model is developed to explore the state-solution space using Q -learning [18]. However, the surrogate model requires a quite long training time and presents low prediction accuracy. To improve prediction accuracy on a small dataset, the prediction problem is converted into a binary classification problem and solved using a support vector classifier, which divides solutions into positive and negative two classes based

on future rewards [19]. A crossover operator is designed to generate decision solutions by combining current fitness and future rewards. The support vector classifier works well on discrete dependence patterns but poorly on other patterns.

According to the above discussion, a new efficient method is necessary to solve dynamic time-linkage optimization problems. As reviewed above, the existing methods are limited in reward prediction based on a small dataset. Indeed, solutions can be selected based on the relative relationship between them rather than the absolute rewards. Based on the relative relationship between any two solutions, a larger dataset can be constructed using historical decisions for model training. Thus, this article proposes a contrastive-learning-based decision making (CLDM) method, which adopts a contrastive model to learn the relationship between solutions in terms of current fitness and future rewards for solution selection. In CLDM, to determine whether to introduce future rewards, a clustering-based time-linkage detection (CD) strategy is adopted to check the intensity of time linkages. Historical decisions are paired to construct contrastive samples as training data. A contrastive model is trained to learn the relationship between solutions. Based on the contrastive model, a quick decision strategy is developed to filter most candidate solutions through clustering and preserve a few ones only for pairwise comparisons. In this way, a decision is quickly made based on the current fitness and future rewards. Since particle swarm optimization (PSO) [20] is efficient, it is taken as a base optimizer to find candidate solutions. Integrating CLDM into PSO, a new algorithm, i.e., contrastive-learning-based PSO (CL-PSO), is put forward. The recent dynamic time-linkage optimization benchmark [18] is taken to construct multiple instances with different dependence patterns for test. The experimental results show that the proposed method performs significantly better than state-of-the-art algorithms on most instances. Each component of CL-PSO helps improve algorithm performance. The contributions of this work can be summarized as follows.

- 1) A contrastive model is developed to learn the relationship between solutions in terms of current fitness and future rewards using limited historical decisions. Compared with prediction models for absolute rewards, the contrastive model can be trained using a larger dataset to improve prediction accuracy.
- 2) A quick decision strategy is proposed to select decision solutions through a few pairwise comparisons only, beneficial to reducing computational time.

The rest of this article is organized as follows. Section II introduces the background. Section III introduces the proposed method. Section IV presents the experimental results. Finally, conclusions are drawn in Section V.

II. BACKGROUND

A. Dynamic Time-Linkage Optimization

In dynamic time-linkage optimization, the environmental state depends on historical decisions and changes with time. A dynamic time-linkage optimization problem can be formulated as

$$\max \sum_{t=0}^{t^{\text{end}}} f(\mathbf{x}_t, \gamma(X_{t-1}^*))$$

$$\text{s.t. } \mathbf{x}_t \in \Gamma \quad (1)$$

where t is the environmental index, t^{end} is the maximum environment index, $\mathbf{x}_t$ is a solution in the decision space Γ of the environment t , X_{t-1}^* is the set of decision solutions in the previous environments, X_{-1}^* is an empty set, and γ is a function related to historical decision solutions and represents the influence of decisions on future states. The optimization objective of the problem is to maximize the cumulative fitness value of decision solutions in all environments.

B. Related Work

In dynamic optimization problems, the optimal solutions change with time [21]. Once the environment changes, a converged population may lose the exploration ability and fail to find new optima [22]. To increase population diversity, new solutions are introduced into the population, which are generated randomly or selected from historical archives [23]. In addition, solution diversity can be maintained along the whole evolutionary procedure through different strategies, such as multipopulation mechanisms [24], quantum operators [25], and multistrategy schemes [26].

Since a new environment is usually relevant to the preceding one, historical solutions can be reused to accelerate algorithm convergence. Based on historical solutions, multiple prediction methods are proposed to learn the changing patterns of optima for predicting new solutions. For example, Kalman filter [27] and linear regression model [28] are adopted to predict new optima. Multilevel prediction is further developed to improve prediction accuracy on nonlinear changing patterns [29]. In addition, by modeling historical optima as time series [30], an autoregression model is adopted to learn the changing patterns [31]. Similarly, recurrent neural networks are used to predict new optima according to multiple historical optimal solutions. These methods use historical information only and work well on stable changing patterns.

To adapt algorithms to arbitrary changes, transfer learning techniques are adopted to learn the transfer model between the new environment and the preceding one [32]. Transfer component analysis [33] and Gaussian process [34] are used to predict the fitness function of new environments. To use information from earlier environments, an incremental support vector machine is developed to implement online evaluation [35]. Similarly, solutions are divided into two groups based on their fitness to train a naive Bayesian classifier for solution evaluation [36]. Instead of predicting the fitness value, to generate high-quality solutions in decision space, neural networks are used to learn the transformation relationship between solutions in neighboring environments [8]. To reduce the computational cost, the solutions with the manifold features are preserved to predict new solutions using manifold transfer learning [37]. Subspace distribution alignment is adopted to transfer historical solutions to new ones using lightweight matrix mapping [38]. Instead of predicting solutions, a denoising autoencoder is used to predict evolution directions to assist the optimization of new environments [39].

To introduce the future rewards of solutions into decision making, prediction models are built to learn the fitness

function of future environments [14]. Based on the future rewards, a probability distribution is calculated for solution selection [15]. To select solutions based on long-term rewards of solutions, reinforcement learning is adopted to learn a policy network for decision making [18]. Similarly, a surrogate model based on Q -learning is developed to select decision solutions based on the environmental state and long-term rewards [18]. Particularly, one Q model is built in each environment and updated in new environments for ensemble evaluation, resulting in the increment of training time of models as time progresses. In addition, the prediction accuracy is limited by the size of historical decisions, which is equal to the number of changes. To reduce the complexity of prediction models, a support vector classifier is proposed to predict whether a solution will bring positive or negative influence on future environments [19]. Candidate solutions are clustered using the classifier and the best solution in the positive class is selected for crossover to generate decision solutions. This method simplifies the evaluation problem to a binary classification problem, which is easier to solve and requires shorter computational time. However, this method evaluates the future rewards in a coarse-grained scheme and meets challenges on complex time-linkage patterns.

III. PROPOSED METHOD

In CL-PSO, a swarm is randomly initialized to search for candidate solutions and CLDM is used for decision making. CLDM adopts a CD strategy to measure the time-linkage intensity. If the linkage is strong, then a contrastive model is trained for solution comparisons and quick decision is adopted for solution selection; otherwise, the best solution found by the swarm is taken as the decision solution. The flowchart of CL-PSO is illustrated in Fig. 1. The details of the components of CL-PSO are introduced in the following.

A. Particle Swarm Optimization

Denote the current environment index as t . PSO is adopted to search for the best solution based on the fitness function. A swarm is randomly initialized with multiple particles. Each particle i owns a position $\mathbf{x}_i$ and velocity $\mathbf{v}_i$. The historical best position found by particle i is recorded as $\mathbf{pbest}_i$. The best position found by the swarm is denoted as the global best position $\mathbf{gbest}$. In each iteration, each particle i updates its position by learning from $\mathbf{gbest}$ and $\mathbf{pbest}_i$ as

$$\begin{aligned} \mathbf{v}_i &= \omega \mathbf{v}_i + c_1 \mathbf{r}_1 (\mathbf{gbest} - \mathbf{x}_i) + c_2 \mathbf{r}_2 (\mathbf{pbest}_i - \mathbf{x}_i) \\ \mathbf{x}_i &= \mathbf{x}_i + \mathbf{v}_i \end{aligned} \quad (2)$$

where ω is the inertia weight, c_1 and c_2 are acceleration factors, and $\mathbf{r}_1$ and $\mathbf{r}_2$ are random vectors in a range of $[0, 1]$. If the new position $\mathbf{x}_i$ is better than $\mathbf{pbest}_i$, then $\mathbf{pbest}_i$ is updated as $\mathbf{x}_i$. If $\mathbf{x}_i$ is better than $\mathbf{gbest}$, then $\mathbf{gbest}$ is updated as $\mathbf{x}_i$. Through multiple iterations, the swarm approximates toward the global optima. The solutions found by the swarm are collected in a set $P_t = \{(\mathbf{x}_1^t, f_1^t), (\mathbf{x}_2^t, f_2^t), \dots, (\mathbf{x}_n^t, f_n^t)\}$ as candidate solutions, where $\mathbf{x}_n^t$ is the n th solution, f_n^t is the fitness value of $\mathbf{x}_n^t$, and n is the number of solutions. The best solution in the current environment is denoted as $\mathbf{x}_t^g$ and its

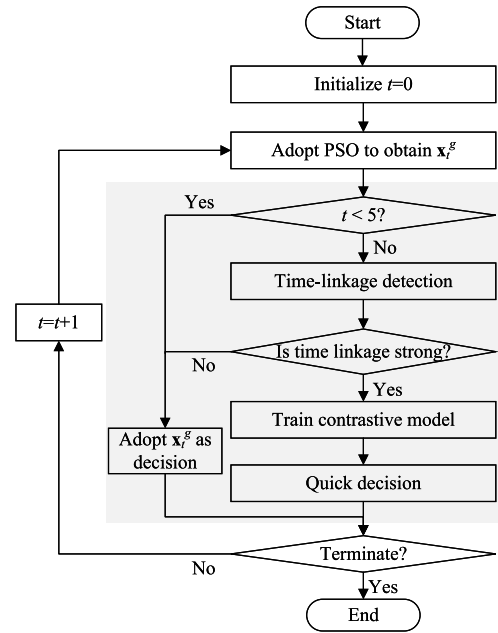


Fig. 1. Flowchart of the proposed CL-PSO, in which the gray part is the CLDM method.

fitness value is denoted as f_t^g . In the first five environments ($t < 5$), $\mathbf{x}_t^g$ is directly taken as the decision solution. These decision solutions are used to observe the impact of solutions on future environments.

B. Clustering-Based Time-Linkage Detection

To detect the time-linkage intensity between environments, decision solutions in historical environments are collected. Since there are usually irrelevant variables, irrelevant variables are removed to improve detection accuracy.

1) *Dataset Construction*: Since the time-linkage feature indicates that a current decision will influence the future state of the environment, decision data and the next environmental states are collected as a dataset. Particularly, the state of an environment can be represented by the fitness value of the optimum. Thus, the decision item at environment j is denoted as $(\mathbf{x}_j^*, f_{j+1}^g)$, where $\mathbf{x}_j^* = (x_{j1}^*, x_{j2}^*, \dots, x_{jD}^*)$ is the decision solution and f_{j+1}^g is the best fitness value found in the next $(j+1)$ th environment. Note that f_{j+1}^g is called the future reward of $\mathbf{x}_j^*$ in the following. The decision items in all historical environments are collected in a set $H_t = \{(\mathbf{x}_0^*, f_1^g), (\mathbf{x}_1^*, f_2^g), \dots, (\mathbf{x}_{t-1}^*, f_t^g)\}$.

2) *Relevant Variable Selection*: The relationship between decision solutions and the state of the next environment is defined to estimate the time-linkage intensity. Dimensionality reduction is performed to improve detection accuracy using the most relevant variables only. To achieve this, the decision items in H_t are divided into two clusters based on the next environmental states. The cluster with higher fitness values is denoted as C_1 with a label 1 and another one is denoted as C_0 with a label of 0. The relevance between variables and the cluster label is evaluated using the mutual information coefficient (MIC) [40]. If the MIC value of a variable x_i is greater than a predefined threshold δ_1 , then x_i is considered as

Algorithm 1 Clustering-based Time-linkage Detection

Input: $H_t = \{(\mathbf{x}_0^*, f_1^g), (\mathbf{x}_1^*, f_2^g), \dots, (\mathbf{x}_{t-1}^*, f_t^g)\}, \delta_2, k$
Output: $flag$

- 1: Select relevant variables using **Algorithm S1**
- 2: $f_{max}^g \leftarrow$ The maximum future reward in H_t
- 3: $f_{min}^g \leftarrow$ The minimum future reward in H_t
- 4: $H' \leftarrow \emptyset$
- 5: **for each** $(\mathbf{x}_i^*, f_{i+1}^g) \in H_t$ **do**
- 6: $\mathbf{x}_i^r \leftarrow$ dimensionality reduction of $\mathbf{x}_i^*$ using relevant variables
- 7: $f_{i+1}^{g'} \leftarrow \frac{f_{i+1}^g - f_{min}^g}{f_{max}^g - f_{min}^g}$
- 8: $H' \leftarrow H' \cup \{(\mathbf{x}_i^r, f_{i+1}^{g'})\}$
- 9: **end for**
- 10: $S \leftarrow \emptyset$
- 11: Divide H' into k clusters based on the variables
- 12: **for each cluster** C_i **do**
- 13: Calculate s_i according to (3)
- 14: $S \leftarrow S \cup \{s_i\}$
- 15: **end for**
- 16: Calculate d_t according to (4)
- 17: **if** $d_t > \delta_2$ **then**
- 18: $flag \leftarrow \text{true}$
- 19: **else**
- 20: $flag \leftarrow \text{false}$
- 21: **end if**

a relevant variable and collected into a set I_r . The procedure of the relevant variable selection is given in Algorithm S1 of the supplementary material.

3) *Clustering-Based Detection*: The decision items in H_t are reduced using the relevant variables and divided into multiple clusters using k -means clustering. In this way, neighboring solutions are grouped into the same cluster. For i th cluster, the mean value s_i of the future rewards of decision items is calculated as

$$s_i = \frac{1}{q} \sum_{j=1}^q \frac{f_j^g - f_{min}^g}{f_{max}^g - f_{min}^g} \quad (3)$$

where q is the number of items in i th cluster, f_j^g is the future reward of the j th item, and f_{min}^g and f_{max}^g are the minimum and maximum values of future rewards among all historical decision items, respectively. Then the time-linkage intensity is calculated according to the difference between any two clusters as

$$d_t = \frac{2}{k^2 - k} \sum_{i=1}^k \sum_{j=i+1}^k |s_i - s_j| \quad (4)$$

where k is the number of clusters. If d_t exceeds a predefined threshold δ_2 , the time linkage is considered as strong and a contrastive model will be adopted for decision making. Otherwise, the current fitness is the most important measure, and the best solution found by PSO is taken as the decision solution. The pseudocode of the detection procedure is given in Algorithm 1.

C. Contrastive Learning Model

A contrastive model is built for solution comparison in terms of current fitness and future rewards. Training data are first constructed to represent the relationship between decisions. Then the model is trained to obtain the optimal parameters.

1) *Dataset Construction*: Historical decision solutions and their next environmental states are collected to construct a dataset $A_t = \{(\mathbf{x}_0^*, f_0^g, f_1^g), (\mathbf{x}_1^*, f_1^g, f_2^g), \dots, (\mathbf{x}_{t-1}^*, f_{t-1}^g, f_t^g)\}$, where $(\mathbf{x}_i^*, f_i^g, f_{i+1}^g)$ is the decision item of the i th environment. The quality of a decision solution is evaluated as the sum of its current fitness and future reward, i.e., $r_i^* = f_i^g + f_{i+1}^g$ for $\mathbf{x}_i^*$ in the i th environment. Any two items in A_t are paired to construct a contrastive sample $(\|\mathbf{x}_i^*, f_i^g, \mathbf{x}_j^*, f_j^g\|, (r_i^* - r_j^*, r_j^* - r_i^*))$, where $\|\mathbf{x}_i^*, f_i^g, \mathbf{x}_j^*, f_j^g\|$ is the input vector, $\|$ is the splice operator, r_i^* is the quality of $\mathbf{x}_i^*$, r_j^* is the quality of $\mathbf{x}_j^*$, and $(r_i^* - r_j^*, r_j^* - r_i^*)$ is the output vector that represents the difference between two decisions. The sample size is $t^2 - t$. To reduce the model size and training difficulty, the inputs of training samples are reduced using the relevant variables defined in Section III-B. In particular, the dimensions related to fitness are preserved to involve the fitness information. Denote the number of relevant variables as d . The dimension of sample inputs is reduced to $2d + 2$. The training samples are normalized into a range of $[-1, 1]$ using the bound of the decision space and the fitness values.

2) *Contrastive Model*: To learn the contrastive relationship between decision solutions, a three-layer radial basis function network (RBFN) [41] is adopted to build a contrastive model as shown in Fig. 2. The first layer is the input layer with $2d + 2$ neurons, and the hidden layer includes m neurons and adopts the radial basis function as the activation function

$$\phi(\mathbf{x}, \mathbf{c}, \sigma) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{c}\|^2}{2\sigma^2}\right) \quad (5)$$

where $\mathbf{x}$ is the input vector of a neuron, $\mathbf{c}$ is the center of the Gaussian function, $\|\mathbf{x} - \mathbf{c}\|$ is the Euclidean distance between $\mathbf{x}$ and $\mathbf{c}$, and σ is the standard deviation of the Gaussian function to control the changing speed of the function. The output layer involves two neurons with a linear function

$$y_j = \sum_{i=1}^m w_{ij} h_i \quad (6)$$

where y_j is the j th dimensional output of the network, w_{ij} is the connection weight between the i th neuron of the hidden layer and j th neuron of the output layer, and h_i is the output of the i th neuron of the hidden layer. RBFN has shown strong approximation ability to linear or nonlinear functions [42].

3) *Model Training*: The network model involves parameters, i.e., $\mathbf{c}$, σ , and connection weights w_{ij} between hidden and output layers. To obtain the optimal parameters, the contrastive model is trained using the inverse matrix method [43]. Particularly, the centers of the radial basis functions are first obtained using k -means clustering on the training data. That is, the training data are clustered into m clusters, in which the cluster centers are taken to set the value of $\mathbf{c}$. Euclidean distance between any two cluster centers is calculated and the maximum value is used to set the σ value, i.e., $\sigma =$

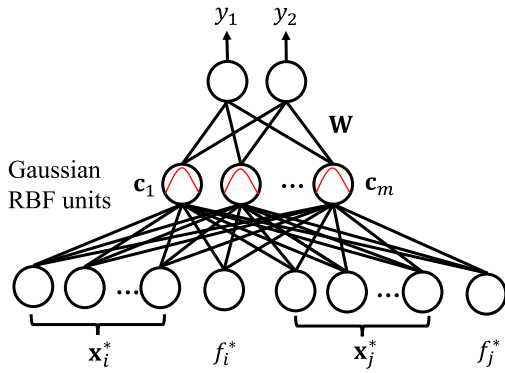


Fig. 2. Network architecture of the contrastive model, which includes one input layer, one hidden layer with m Gaussian RBF units, and one output layer. $\mathbf{W}$ is the connection weight between the hidden and output layers. Two historical decisions, i.e., $\mathbf{x}_i^*$ and $\mathbf{x}_j^*$ with fitness values of f_i^* and f_j^* , are input to the network, and (y_1, y_2) is the network output.

Algorithm 2 Contrastive Learning Model

Input: $A_t = \{(\mathbf{x}_0^*, f_0^*, f_1^g), \dots, (\mathbf{x}_{t-1}^*, f_{t-1}^*, f_t^g)\}$, the number m of neurons in hidden layer

Output: A contrastive model

```

1:  $S \leftarrow \emptyset$ 
2: for  $i = 0$  to  $t - 1$  do
3:   for  $j = i + 1$  to  $t - 1$  do
4:      $r_i^* \leftarrow f_i^* + f_{i+1}^g$ 
5:      $r_j^* \leftarrow f_j^* + f_{j+1}^g$ 
6:      $s \leftarrow (||(\mathbf{x}_i^*, f_i^*, \mathbf{x}_j^*, f_j^*), (r_i^* - r_j^*, r_j^* - r_i^*)$ 
7:      $p \leftarrow (||(\mathbf{x}_j^*, f_j^*, \mathbf{x}_i^*, f_i^*), (r_j^* - r_i^*, r_i^* - r_j^*)$ 
8:      $S \leftarrow S \cup \{s, p\}$ 
9:   end for
10: end for
11: Reduce the dimensionality of samples in  $S$  using relevant variables
12: Normalize the data of  $S$  to the range of  $[-1, 1]$ 
13: if  $m > |S|$  then
14:    $m \leftarrow |S|$ 
15: end if
16: Build a contrastive model with  $m$  hidden neurons
17: Train the network parameters using  $S$ 

```

$(c_{\max}/(2m)^{1/2})$, where $c_{\max}$ is the maximum distance among all the cluster pairs. All the training data are input to the network and the output of the hidden layer is denoted as a matrix $\mathbf{h}$. Then the target output of the training data can be represented by $\mathbf{y} = \mathbf{W}\mathbf{h}$, where $\mathbf{W}$ is the mapping matrix between the hidden layer and the output layer. The optimal weight matrix can be obtained by pseudoinverse matrix fitting, i.e., $\mathbf{W} = \mathbf{y}\mathbf{h}^+$, where $\mathbf{h}^+$ is the pseudoinverse matrix of $\mathbf{h}$. The procedure of the contrastive learning model is given in Algorithm 2.

D. Quick Decision

Using the contrastive model, the solutions in P_t are ranked to find the one with the best value in terms of both current fitness and future rewards. Since the contrastive model requires n^2 comparison times for n solutions, quick decision is developed to reduce the computational time.

Algorithm 3 Quick Decision

Input: $P_t = \{(\mathbf{x}_t^1, f_t^1), \dots, (\mathbf{x}_t^n, f_t^n)\}$, a contrastive model, k

Output: Decision solution $\mathbf{x}_t^*$

```

1:  $Q \leftarrow \emptyset$ 
2: Divide  $P_t$  into  $k$  clusters based on the relevant variables and fitness values
3: for each cluster do
4:   Find the solution  $\mathbf{x}_t^a$  with the maximum fitness value
5:   Randomly select two distant solutions  $\mathbf{x}_t^b$  and  $\mathbf{x}_t^c$ 
6:    $Q \leftarrow Q \cup \{(\mathbf{x}_t^a, f_t^a), (\mathbf{x}_t^b, f_t^b), (\mathbf{x}_t^c, f_t^c)\}$ 
7: end for
8:  $n \leftarrow |Q|$ 
9:  $S \leftarrow \text{zeros}(n)$ 
10: for  $i = 1$  to  $n$  do
11:   for  $j = 1$  to  $n$  do
12:     if  $i \neq j$  then
13:        $y_1, y_2 \leftarrow \text{contrastive model}(\mathbf{x}_t^i, f_t^i, \mathbf{x}_t^j, f_t^j)$ 
14:       if  $y_1 > y_2$  then
15:          $S[i] \leftarrow S[i] + 1$ 
16:       else
17:          $S[j] \leftarrow S[j] + 1$ 
18:       end if
19:     end if
20:   end for
21: end for
22:  $(\mathbf{x}_t^*, f_t^*) \leftarrow \arg \max_{(\mathbf{x}_t^a, f_t^a) \in Q} S[a]$ 
23: Select  $\mathbf{x}_t^*$  as the decision solution

```

Candidate solutions are divided into k clusters using k -means clustering based on relevant variables and current fitness. The best solution and two random solutions are selected from each cluster to form a set Q for comparison. Particularly, the best solution is selected to represent the local optima and random solutions help avoid being trapped in local optima. Solutions in Q are compared in pairs using the contrastive model. Given two solutions $\mathbf{u}$ and $\mathbf{v}$, if the output (y_1, y_2) of the contrastive model satisfies $y_1 > y_2$, then it is considered that $\mathbf{u}$ is better than $\mathbf{v}$; otherwise, $\mathbf{u}$ is worse than $\mathbf{v}$. Based on the comparison results, a directed topology graph can be constructed to represent the relationship between solutions. The best solution can be found by searching the ending vertex of paths in the graph. However, it is time-consuming and may fail in cycles [42]. Thus, we directly calculate the successful times of a solution in pairwise comparisons and select the one with the highest times as the decision solution. The procedure of the quick decision is given in Algorithm 3.

E. Complete Algorithm

Denote the environment index as t and an archive as H_t . The sign t is initialized as 0 and H_t is initialized as an empty set. At the beginning, a swarm is randomly initialized to search for the best solution according to the learning strategy [see (2)] through multiple iterations until reaching the maximum function evaluation times. The solutions found by the swarm along the search procedure are collected in a set P_t for decision solution selection. If the environment index

Algorithm 4 CL-PSO**Input:** A dynamic time-linkage optimization problem**Output:** Decisions of all environments $\{\mathbf{x}_t^* | 0 \leq t \leq t^{end}\}$

```

1:  $t \leftarrow 0$ ,  $H_t \leftarrow \emptyset$ ,  $A_t \leftarrow \emptyset$ 
2: while termination criteria not met do
3:   Perform PSO and collect the solutions found in  $P_t$ 
4:   if  $t < 5$  then
5:     Take the best solution in  $P_t$  as  $\mathbf{x}_t^*$ 
6:   else
7:      $flag \leftarrow$  Clustering-based time-linkage detection
       using Algorithm 1
8:     if  $flag$  is true then
9:       Train a contrastive model using Algorithm 2
10:      Perform quick decision to select  $\mathbf{x}_t^*$  using
        Algorithm 3
11:    else
12:      Take the best solution in  $P_t$  as  $\mathbf{x}_t^*$ 
13:    end if
14:  end if
15:   $f_t^g \leftarrow$  the best fitness value in  $P_t$ 
16:   $f_t^* \leftarrow$  the fitness value of  $\mathbf{x}_t^*$  in the current environment
17:  if  $t > 0$  then
18:     $H_t \leftarrow H_{t-1} \cup \{(\mathbf{x}_{t-1}^*, f_t^g)\}$ 
19:     $A_t \leftarrow A_{t-1} \cup \{(\mathbf{x}_{t-1}^*, f_{t-1}^*, f_t^g)\}$ 
20:  end if
21:   $t \leftarrow t + 1$ 
22: end while

```

$t < 5$, then the best solution found by the swarm is taken as the decision solution; otherwise, the time-linkage detection strategy is performed to check the time-linkage intensity. If the time linkage is weak, then the best solution is selected from P_t as the decision solution; otherwise, a contrastive model is built to train using contrastive samples, which are constructed by pairing any two historical decisions. After that, solutions are divided into multiple clusters using k -means clustering. The best solution and two random solutions are selected from each cluster to perform pairwise comparisons using the contrastive model. The winner is taken as the decision solution and collected to H_t . Once the environment changes, the swarm is reinitialized and a new decision solution is determined using the same procedure. The algorithm continues until reaching the termination conditions. The decision solutions in all environments are output finally. The procedure of the complete algorithm is given in Algorithm 4.

IV. EXPERIMENT

A. Experimental Setting

1) *Test Problems*: Based on the recent dynamic time-linkage optimization benchmark [18], [19], multiple instances with different dependence patterns are constructed by

$$\begin{aligned}
 f(\mathbf{x}, t) &= \max_{i=1}^n \{h_i - w_i \|\mathbf{x} - \mathbf{p}_i\|\} + \gamma(\mathbf{x}_{t-1}^*) \\
 \text{s.t. } \mathbf{x} &\in \Gamma \\
 h_i &= h_i + s_{\text{height}} \times \mathcal{N}(0, 1) \\
 w_i &= w_i + s_{\text{width}} \times \mathcal{N}(0, 1)
 \end{aligned}$$

$$\begin{aligned}
 \mathbf{p}_i &= \mathbf{p}_i + s_{\text{length}} \times \frac{\mathbf{v}_i}{\|\mathbf{v}_i\|} \\
 \mathbf{v}_i &= (1 - \lambda) \times \mathbf{u} + \lambda \times \mathbf{v}_i
 \end{aligned} \quad (7)$$

where $\mathbf{x}$ is a D -dimensional solution in the decision space Γ , n is the number of peaks, t is the environment index, h_i , w_i , and $\mathbf{p}_i$ are, respectively, the height, width, and peak location of the i th peak and change with time, $\gamma(\mathbf{x}_{t-1}^*)$ is a function related to the decision solution $\mathbf{x}_{t-1}^*$ in the preceding environment, s_{height} , s_{width} , and s_{length} are, respectively, the change severity of height, width, and peak locations, $\mathcal{N}(0, 1)$ is a random number generated by the normal distribution with mean of 0 and standard deviation of 1, $\mathbf{v}_i$ is the movement vector of the i th peak location, $\mathbf{u}$ is a random vector in a range of $[0, 1]$, and λ is a parameter to control the changing pattern of peak locations. Denote the lower and upper bounds of the decision space as l and u , respectively. Following [18], [19], discrete, continuous, and disturbance types are combined with linear, sinusoidal, circular, hyperbolic tangent (tanh), and maximum features to form eight functions of $\gamma(\mathbf{x}_{t-1}^*)$ to represent different dependence patterns.

Discrete-linear pattern

$$\gamma(\mathbf{x}_{t-1}^*) = \begin{cases} b, & x_{t-1,1} \geq 0 \\ -b, & x_{t-1,1} < 0. \end{cases} \quad (8)$$

Discrete-sin pattern

$$\gamma(\mathbf{x}_{t-1}^*) = \begin{cases} b, & x_{t-1,1} \geq \frac{u}{2} \sin\left(\frac{\pi}{u} x_{t-1,2}\right) \\ -b, & x_{t-1,1} < \frac{u}{2} \sin\left(\frac{\pi}{u} x_{t-1,2}\right). \end{cases} \quad (9)$$

Discrete-circle pattern

$$\gamma(\mathbf{x}_{t-1}^*) = \begin{cases} b, & (x_{t-1,1})^2 + (x_{t-1,2})^2 \geq \frac{u^2}{2\pi} \\ -b, & (x_{t-1,1})^2 + (x_{t-1,2})^2 < \frac{u^2}{2\pi}. \end{cases} \quad (10)$$

Continuous-linear pattern

$$\gamma(\mathbf{x}_{t-1}^*) = \frac{b}{u} x_{t-1,1}. \quad (11)$$

Continuous-sin pattern

$$\gamma(\mathbf{x}_{t-1}^*) = \frac{b}{u} \left(x_{t-1,1} - \frac{u}{2} \sin\left(\frac{\pi}{u} x_{t-1,2}\right) \right). \quad (12)$$

Continuous-circle pattern

$$\gamma(\mathbf{x}_{t-1}^*) = \frac{b}{u^2} \left((x_{t-1,1})^2 + (x_{t-1,2})^2 \right). \quad (13)$$

Disturbance-tanh pattern

$$\gamma(\mathbf{x}_{t-1}^*) = b \left(\mathcal{N}(0, \sigma_{\text{tanh}}) + \tanh(x_{t-1,1}) \right). \quad (14)$$

Disturbance-max pattern

$$\gamma(\mathbf{x}_{t-1}^*) = b \times \max\{x_{t-1,1}, x_{t-1,2}\} + \mathcal{N}(0, \sigma_{\text{max}}) \quad (15)$$

where b is a constant to control the intensity of time linkages and indicates the influence of decisions on future environments. Particularly, the disturbance patterns incorporate noise to test the robustness of algorithms under nonideal conditions.

Examples of eight dependence patterns in a 2-D decision space are illustrated in Fig. S1 of the supplementary material.

In the experiments, $D = 10$, $l = -5$, $u = 5$, $m = 1$, $s_{\text{height}} = 7$, $s_{\text{width}} = 7$, $s_{\text{length}} = 1$, and $\lambda = 0.5$ following [18]. Each problem instance undergoes 99 environmental changes, with a change frequency of 10000 function evaluations.

2) *Competing Algorithms*: Six state-of-the-art algorithms are selected for comparison, i.e., PSO [44], PSO+Predictor [4], PSO with surrogate-assisted Q -learning (SQL-PSO) [18], PSO with a clustering-based support vector classifier (CSVC-PSO) [19], federated edge learning with Softmax deep double deterministic policy gradient (FL-SD3) [45], and genetic algorithm with the proposed CLDM (CL-GA). Specifically, PSO selects decision solutions based on current fitness. PSO+Predictor uses a combination of quadratic, S-type, and trigonometric functions to construct a predictor for estimating the future rewards of solutions. SQL-PSO adopts reinforcement learning to learn a surrogate model for predicting the long-term rewards of solutions. CSVC-PSO uses a support vector classifier to roughly predict the positive or negative influence of solutions and uses crossover to generate decision solutions. FL-SD3 uses deep reinforcement learning for sequential decision making and uses softmax deep double deterministic policy gradient for optimization. CL-GA integrates the proposed CLDM with a genetic algorithm [46]. In all the algorithms, the population size is set to 100. For PSO, PSO+Predictor, SQL-PSO, CSVC-PSO, and CL-PSO, $\omega = 0.75$, and $c_1 = c_2 = 1.4$ following [18].

3) *Performance Measure*: To evaluate the performance of CL-PSO, the cumulative fitness value of decision solutions in all environments is used as

$$F = \sum_{t=0}^{t^{\text{end}}} f(\mathbf{x}_t^*, t) \quad (16)$$

where $\mathbf{x}_t^*$ is the decision solution in the t th environment.

On each problem instance, each algorithm independently runs 30 times. The mean values and standard deviations of the cumulative fitness are reported. To observe the significance difference, the Wilcoxon rank-sum test is performed between CL-PSO and each competing algorithm at a significance level of 0.05. The significance results are denoted as the signs “+,” “≈,” and “−,” representing that CL-PSO is significantly better than, equal to, or worse than the competing algorithm, respectively. The best result is highlighted in **bold**.

B. Parameter Investigation

1) δ_1 : The threshold δ_1 is adopted to control the selection of relevant variables. The sign δ_1 is tested in a range of (0, 1) with a step size of 0.1. The results of CL-PSO are reported in Tables S.I and S.II of the supplementary material. CL-PSO performs well with $\delta_1 \geq 0.7$. Particularly, CL-PSO with $\delta_1 = 0.8$ performs better than that with 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.9 values on 30, 36, 34, 24, 14, 13, 1, and 1 out of 36 instances, whereas worse on 0, 0, 0, 3, 1, 3, 0, and 0 instances. This shows that $\delta_1 = 0.8$ performs better on most instances. Thus, δ_1 is set as 0.8.

2) δ_2 : The threshold δ_2 is used to control the detection of the time-linkage intensity, which determines the decision making using PSO or the contrastive model. Taking six instances with continuous-linear and discrete-linear dependence patterns under $b = 10, 50, 100$ as examples, δ_2 is tested in a range of [0, 1] with a step size of 0.2. Fig. S2 illustrates the box plots of the cumulative fitness obtained by CL-PSO. CL-PSO performs the best with $\delta_2 = 0.2$ in almost all the instances. The performance of CL-PSO sharply degenerates using a large δ_2 value especially when $\delta_2 \geq 0.6$. This is because a large δ_2 value may wrongly filter strong-linkage cases, resulting in the neglect of future rewards of decisions. Thus, $\delta_2 = 0.2$ is a good choice.

3) k : In both the time-linkage detection and quick decision strategies, solutions are divided into k clusters. The k value is set as 4, 8, 16, 32, and 64, respectively. Taking instances with continuous-linear, continuous-sin, continuous-circle, discrete-linear, discrete-sin, and discrete-circle patterns under $b = 0, 20, 40, 60, 80, 100$ as examples, the cumulative fitness values of CL-PSO with different k values are reported in Table S.III of the supplementary material. A too high or too low k value causes poor performance. CL-PSO with $k = 16$ performs the best on 24 out of 36 instances. Therefore, k is set as 16.

4) m : The number m of nodes in the hidden layer is set as 4, 8, 16, 32, and 64 for test on 36 instances with continuous-linear, continuous-sin, continuous-circle, discrete-linear, discrete-sin, and discrete-circle patterns under $b = 0, 20, 40, 60, 80, 100$. The results of CL-PSO are listed in Table S.IV of the supplementary material. Particularly, $m = 16$ performs the best on 16 out of 36 instances. In addition, $m = 16$ performs better than $m = 4, 8, 32$, and 64 on 27, 16, 12, and 23 out of 36, whereas worse on zero, four, six, and two instances. Thus, m is set as 16.

C. Effects of Components in CL-PSO

CL-PSO consists of three new components, i.e., CD, contrastive learning model, and quick decision.

1) *Clustering-Based Time-Linkage Detection*: To verify the effect of the clustering-based time-linkage detection (CD), the binary-classifier-based detection (BD) strategy in [19] is adopted for comparison. In addition, a CL-PSO variant without the detection strategy (noCD) is compared. Thirty-six instances with different types and intensities of dependence patterns are constructed for test. The results of CD, BD, and noCD are reported in Table I. CD performs significantly better than BD and noCD on 26 and 17 out of 36 instances, whereas worse on one and eight instances only. CD performs the best on 24 instances, followed by noCD and BD with ten and seven instances. Specially, BD performs well on some instances with discrete patterns due to the binary property of these instances. In contrast, CD performs better than BD in almost all the instances with continuous dependence patterns, showing that the CD strategy can better detect complex dependence patterns.

To further observe the detection accuracy of the CD strategy, Fig. S3 illustrates the mean intensity $[d_t$ in (4)] of the time-linkage feature obtained by the detection strategy

TABLE I

CUMULATIVE FITNESS OF CL-PSO WITH DIFFERENT DETECTION STRATEGIES AND THE VARIANT WITHOUT DETECTION STRATEGIES

Mode	Pattern	b	BD	noCD	CD		
Linear		0	5268.53 $\pm$ 0.01 ($\approx$)	4959.21 $\pm$ 83.09 (+)	5268.52 $\pm$ 0.01		
		20	5494.52 $\pm$ 0.02 (+)	5453.88 $\pm$ 51.87 (+)	5503.55 $\pm$ 8.00		
		40	5720.53 $\pm$ 0.02 (+)	6315.79 $\pm$ 29.27 ($-$)	6120.44 $\pm$ 24.20		
		60	5954.21 $\pm$ 3.28 (+)	7477.26 $\pm$ 27.30 ($-$)	7124.07 $\pm$ 63.02		
		80	7320.80 $\pm$ 119.37 (+)	8768.42 $\pm$ 52.87 ($-$)	8339.45 $\pm$ 61.35		
		100	8622.53 $\pm$ 167.42 (+)	10112.18 $\pm$ 62.34 ($-$)	9566.40 $\pm$ 49.78		
	Continuous	Sin	0	5268.52 $\pm$ 0.01 ($\approx$)	4973.37 $\pm$ 69.06 (+)	5268.52 $\pm$ 0.01	
			20	5067.74 $\pm$ 0.01 ($-$)	4895.74 $\pm$ 66.65 (+)	5035.92 $\pm$ 24.84	
			40	4866.98 $\pm$ 0.02 (+)	5189.89 $\pm$ 111.69 (+)	5411.00 $\pm$ 61.35	
			60	5154.92 $\pm$ 697.73 (+)	6029.86 $\pm$ 212.23 ($\approx$)	6096.16 $\pm$ 153.60	
	Circle	80	6410.84 $\pm$ 893.66 ($\approx$)	6847.84 $\pm$ 296.38 ($\approx$)	6976.49 $\pm$ 192.72		
		100	7890.70 $\pm$ 852.82 ($\approx$)	7986.84 $\pm$ 344.30 ($\approx$)	7933.69 $\pm$ 281.82		
				0	5268.52 $\pm$ 0.01 ($\approx$)	4960.73 $\pm$ 61.77 (+)	5268.52 $\pm$ 0.01
				20	6485.76 $\pm$ 0.02 (+)	6351.85 $\pm$ 91.69 (+)	6487.02 $\pm$ 1.67
40	7703.01 $\pm$ 0.03 (+)			7831.84 $\pm$ 115.98 ($-$)	7725.45 $\pm$ 5.39		
60	8920.25 $\pm$ 0.04 (+)			9312.87 $\pm$ 183.97 ($-$)	8981.93 $\pm$ 25.57		
80	10137.49 $\pm$ 0.05 (+)			10962.46 $\pm$ 194.14 ($-$)	10277.28 $\pm$ 78.08		
100	11354.74 $\pm$ 0.08 (+)			12704.08 $\pm$ 196.00 ($-$)	11617.52 $\pm$ 169.87		
Linear		0	5268.52 $\pm$ 0.01 ($\approx$)	4957.37 $\pm$ 71.62 (+)	5268.52 $\pm$ 0.01		
		20	5688.52 $\pm$ 0.01 (+)	5891.04 $\pm$ 108.34 (+)	5921.23 $\pm$ 53.86		
		40	7063.48 $\pm$ 225.39 (+)	7496.40 $\pm$ 145.04 (+)	7582.10 $\pm$ 97.64		
		60	8460.59 $\pm$ 519.58 (+)	9083.15 $\pm$ 193.57 ($\approx$)	9054.94 $\pm$ 98.33		
		80	10486.14 $\pm$ 371.86 (+)	10732.05 $\pm$ 248.12 ($\approx$)	10741.56 $\pm$ 79.29		
		100	12159.64 $\pm$ 452.21 (+)	12374.96 $\pm$ 301.82 ($\approx$)	12375.42 $\pm$ 67.24		
	Discrete	Sin	0	5268.52 $\pm$ 0.01 ($\approx$)	4944.22 $\pm$ 76.91 (+)	5268.52 $\pm$ 0.01	
			20	5328.52 $\pm$ 0.01 (+)	5210.59 $\pm$ 100.48 (+)	5391.51 $\pm$ 65.54	
			40	5388.52 $\pm$ 0.01 (+)	6564.23 $\pm$ 277.29 (+)	6691.21 $\pm$ 209.31	
			60	7488.84 $\pm$ 1042.09 (+)	8192.45 $\pm$ 265.89 ($\approx$)	8284.28 $\pm$ 235.37	
	Circle	80	8372.82 $\pm$ 1728.92 (+)	9917.54 $\pm$ 322.85 ($\approx$)	9992.55 $\pm$ 271.08		
		100	10437.70 $\pm$ 2164.77 ($\approx$)	11576.74 $\pm$ 524.82 ($\approx$)	11692.89 $\pm$ 276.04		
				0	5268.52 $\pm$ 0.01 ($\approx$)	4957.80 $\pm$ 97.38 (+)	5268.52 $\pm$ 0.01
				20	5488.52 $\pm$ 0.01 (+)	5454.91 $\pm$ 129.08 (+)	5593.65 $\pm$ 56.16
40	5682.52 $\pm$ 25.91 (+)			6180.97 $\pm$ 213.95 (+)	6416.85 $\pm$ 138.71		
60	5890.48 $\pm$ 128.18 (+)			7007.85 $\pm$ 368.11 (+)	7327.82 $\pm$ 276.00		
80	6086.92 $\pm$ 179.68 (+)			7924.34 $\pm$ 489.17 ($\approx$)	8199.71 $\pm$ 312.77		
100	6294.24 $\pm$ 160.46 (+)			9056.76 $\pm$ 542.02 ($\approx$)	9073.40 $\pm$ 443.85		
+ / $\approx$ / -			26 / 9 / 1	17 / 11 / 8			
Best			7	10	24		

over all environments. As shown in Fig. S3, the intensity values of both CD and BD increase with the growth of b values, showing that the intensity measure can well reflect the time-linkage property. Based on the predefined threshold, the BD strategy only takes instances with $b \geq 60$ as strong time-linkage cases and presents high detection errors on other instances. In contrast, the CD strategy takes most instances with $b > 20$ as strong time-linkage cases. In this way, the contrastive model can be used for decision making to improve algorithm performance.

2) *Contrastive Learning Model*: The contrastive model aims to obtain the ranking relationship of solutions through pairwise comparisons. The results of pairwise comparisons at each environment are recorded and measured using two indices, i.e., accuracy and Spearman's rank correlation coefficient (Spearman's ρ) [47]. Particularly, the accuracy index evaluates the ratio of the number of correct relationships obtained by the contrastive model; Spearman's ρ index evaluates the similarity between the ranking obtained by the contrastive model and the real one. Taking six instances with different dependence patterns under $b = 100$ as examples, Figs. 3 and S4 of the supplementary material illustrate the mean values of accuracy and Spearman's ρ over 30 runs in each environment. In all the instances, both the accuracy and Spearman's ρ present an upward trend as the environment index increases. This is because as time progresses, the growing set of decision data enables the algorithm to train a contrastive model with higher accuracy. Particularly, the accuracy is larger than 0.5 in almost all environments and even increases to 0.8 later. In addition, the values of accuracy and Spearman's ρ are nearly 1.0 on instances with

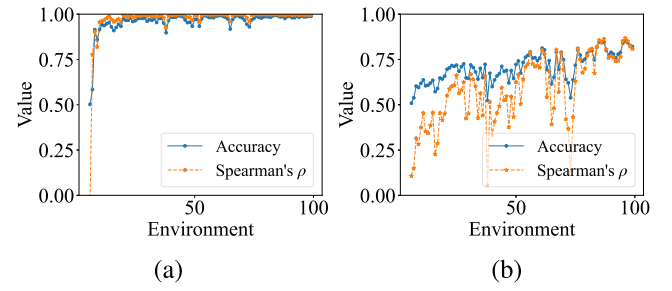


Fig. 3. Accuracy and Spearman's ρ of the contrastive model on instances with (a) continuous-linear and (b) discrete-linear dependence patterns under $b = 100$.

the continuous-linear dependence pattern. Thus, the contrastive model can well predict the ranking relationships among solutions to assist decision making.

3) *Quick Decision*: The quick decision strategy filters poor solutions using k -means clustering to reduce the computational time. Taking 12 instances with different types and intensities of dependence patterns, the mean computational time of CL-PSO and the variant without clustering (noClustering) in one environment is reported in Table S.V of the supplementary material. On all the instances, the computational time of CL-PSO is in a range of [0.49, 0.60] s. In contrast, noClustering requires longer than 735 s, which is thousands of times to that of CL-PSO. This is because the number of pairwise comparisons is a quadratic function of that of candidate solutions. Thus, the clustering operator can significantly reduce the computational time.

From each cluster, three solutions, i.e., the best and two random ones, are selected for decision making. To test the effect of random solutions, one CL-PSO variant without using random solutions, named noRS, is compared. Instances with different types and intensities of dependence patterns are taken for test. As shown in Table S.VI of the supplementary material, CL-PSO performs better than noRS in ten instances, whereas worse on three instances only. CL-PSO performs the best on 22 out of 36 instances. Thus, random solutions are necessary.

D. Comparison to Other Algorithms

1) *Results on 10-D Instances*: Forty instances with eight dependence patterns are constructed for the test. For each dependence pattern, five instances are constructed with different time-linkage intensities, i.e., $b = 10, 30, 50, 80, 100$. The cumulative fitness of all the algorithms is reported in Table II. CL-PSO performs better than PSO, PSO+Predictor, SQL-PSO, CSVC-PSO, FL-SD3, and CL-GA on 34, 34, 32, 34, 40, and 22 out of 40 instances, whereas worse on four, four, eight, four, zero, and 13 instances only. In addition, CL-PSO performs the best on 18 instances, followed by CL-GA and CSVC-PSO with 15 and six instances.

CL-PSO performs similar to PSO on instances with $b = 10$, since the time linkage is detected to be weak and PSO is directly adopted for decision making. On instances with strong time linkages ($b \geq 30$), CL-PSO outperforms PSO on almost all the instances, showing that the contrastive model can well predict the current fitness and future rewards for decision making. Compared with PSO+Predictor, CL-PSO performs better on all the instances except four instances

TABLE II
RESULTS OF PSO, PSO+PREDICTOR, SQL-PSO, CSVC-PSO, FL-SD3, CL-GA AND CL-PSO ON 10-D INSTANCES

Mode	Pattern	b	PSO [44]	PSO+Predictor [4]	SQL-PSO [18]	CSVC-PSO [19]	FL-SD3 [45]	CL-GA	CL-PSO
Continuous	Linear	10	5381.52 $\pm$ 0.01 (–)	2783.39 $\pm$ 158.58 (+)	3920.06 $\pm$ 631.85 (+)	5381.53 $\pm$ 0.01 (–)	1594.32 $\pm$ 266.01 (+)	4998.61 $\pm$ 54.65 (+)	5380.26 $\pm$ 1.31
		30	5607.52 $\pm$ 0.02 (+)	2933.41 $\pm$ 122.00 (+)	3959.92 $\pm$ 745.83 (+)	5607.52 $\pm$ 0.02 (+)	1729.56 $\pm$ 272.66 (+)	5365.82 $\pm$ 86.85 (+)	5730.92 $\pm$ 10.17
		50	5833.52 $\pm$ 0.02 (+)	3604.58 $\pm$ 136.74 (+)	4427.43 $\pm$ 727.90 (+)	5833.53 $\pm$ 0.02 (+)	1751.50 $\pm$ 343.97 (+)	7059.20 $\pm$ 110.82 (–)	6610.61 $\pm$ 48.02
		80	6172.52 $\pm$ 0.04 (+)	5490.14 $\pm$ 131.37 (+)	4376.14 $\pm$ 985.24 (+)	6341.84 $\pm$ 39.64 (+)	1877.29 $\pm$ 284.61 (+)	9460.48 $\pm$ 120.15 (–)	8330.37 $\pm$ 70.70
		100	6398.53 $\pm$ 0.05 (+)	7003.17 $\pm$ 145.40 (+)	5157.64 $\pm$ 943.48 (+)	6953.89 $\pm$ 48.75 (+)	1849.66 $\pm$ 454.92 (+)	10994.67 $\pm$ 120.09 (–)	9554.02 $\pm$ 57.07
	Sin	10	5168.13 $\pm$ 0.02 (–)	2978.94 $\pm$ 175.42 (+)	3863.31 $\pm$ 465.72 (+)	5168.13 $\pm$ 0.01 (–)	1605.29 $\pm$ 201.73 (+)	4954.03 $\pm$ 81.81 (+)	5130.60 $\pm$ 14.74
		30	4967.36 $\pm$ 0.02 (+)	3516.44 $\pm$ 149.20 (+)	3687.10 $\pm$ 619.81 (+)	4967.36 $\pm$ 0.02 (+)	1471.94 $\pm$ 225.54 (+)	5114.58 $\pm$ 178.44 ($\approx$)	5159.37 $\pm$ 57.00
		50	4766.58 $\pm$ 0.03 (+)	4325.81 $\pm$ 165.25 (+)	4160.08 $\pm$ 1091.14 (+)	4766.60 $\pm$ 0.03 (+)	1489.34 $\pm$ 220.32 (+)	6766.95 $\pm$ 261.16 (–)	5742.81 $\pm$ 83.22
		80	4465.42 $\pm$ 0.05 (+)	6347.21 $\pm$ 341.18 (+)	4718.46 $\pm$ 1279.04 (+)	4684.09 $\pm$ 68.21 (+)	1379.72 $\pm$ 328.44 (+)	9343.28 $\pm$ 266.11 (–)	7033.08 $\pm$ 261.37
		100	4264.66 $\pm$ 0.07 (+)	8074.15 $\pm$ 372.63 (–)	5243.02 $\pm$ 1374.65 (+)	5146.11 $\pm$ 118.45 (+)	1257.22 $\pm$ 342.21 (+)	10974.07 $\pm$ 583.98 (–)	7961.15 $\pm$ 220.79
	Circle	10	5877.14 $\pm$ 0.01 ($\approx$)	3274.51 $\pm$ 206.19 (+)	4475.23 $\pm$ 642.47 (+)	5877.14 $\pm$ 0.01 ($\approx$)	1747.53 $\pm$ 215.95 (+)	5662.88 $\pm$ 36.34 (+)	5877.14 $\pm$ 0.01
		30	7094.39 $\pm$ 0.02 (+)	5489.58 $\pm$ 172.33 (+)	6104.34 $\pm$ 559.63 (+)	7094.38 $\pm$ 0.03 (+)	2152.17 $\pm$ 353.74 (+)	6931.97 $\pm$ 79.49 (+)	7102.81 $\pm$ 2.46
		50	8311.63 $\pm$ 0.03 (+)	8313.27 $\pm$ 463.85 ($\approx$)	7215.31 $\pm$ 808.58 (+)	8311.64 $\pm$ 0.03 (+)	2495.98 $\pm$ 351.74 (+)	8448.94 $\pm$ 308.96 ($\approx$)	8351.88 $\pm$ 12.58
		80	10137.51 $\pm$ 0.05 (+)	13756.10 $\pm$ 747.40 (–)	8538.88 $\pm$ 1116.25 (+)	10137.49 $\pm$ 0.05 (+)	3022.96 $\pm$ 497.58 (+)	11180.29 $\pm$ 827.96 ($\approx$)	10243.04 $\pm$ 85.27
		100	11354.75 $\pm$ 0.05 (+)	16979.72 $\pm$ 1203.95 (–)	10166.36 $\pm$ 1040.38 (+)	11354.75 $\pm$ 0.07 (+)	3530.31 $\pm$ 635.81 (+)	13179.43 $\pm$ 1041.03 (–)	11549.52 $\pm$ 95.09
Discrete	Linear	10	5478.52 $\pm$ 0.01 ($\approx$)	2761.36 $\pm$ 247.02 (+)	3879.64 $\pm$ 578.33 (+)	5478.52 $\pm$ 0.01 ($\approx$)	1593.81 $\pm$ 278.90 (+)	4888.92 $\pm$ 166.73 (+)	5477.34 $\pm$ 21.65
		30	5898.52 $\pm$ 0.01 (+)	3233.59 $\pm$ 210.82 (+)	4270.94 $\pm$ 826.91 (+)	5898.52 $\pm$ 0.01 (+)	1778.24 $\pm$ 293.75 (+)	5887.45 $\pm$ 131.04 (+)	6758.19 $\pm$ 102.36
		50	6318.52 $\pm$ 0.01 (+)	4240.14 $\pm$ 356.86 (+)	5181.98 $\pm$ 1169.50 (+)	7707.84 $\pm$ 106.31 (+)	1820.95 $\pm$ 427.50 (+)	7496.02 $\pm$ 207.24 (+)	8335.99 $\pm$ 59.59
		80	6948.52 $\pm$ 0.01 (+)	6599.65 $\pm$ 439.19 (+)	7618.98 $\pm$ 1521.69 (+)	9366.81 $\pm$ 181.86 (+)	2074.32 $\pm$ 517.97 (+)	10100.54 $\pm$ 319.72 (+)	10769.05 $\pm$ 70.34
		100	7368.52 $\pm$ 0.01 (+)	7973.28 $\pm$ 558.20 (+)	9305.65 $\pm$ 1286.41 (+)	10481.02 $\pm$ 209.65 (+)	2124.36 $\pm$ 630.01 (+)	11655.93 $\pm$ 513.81 (+)	12390.03 $\pm$ 69.18
	Sin	10	5298.52 $\pm$ 0.01 (–)	2689.60 $\pm$ 202.23 (+)	3900.27 $\pm$ 641.67 (+)	5298.52 $\pm$ 0.01 (–)	1589.60 $\pm$ 223.34 (+)	4959.23 $\pm$ 167.47 (+)	5296.35 $\pm$ 1.09
		30	5358.52 $\pm$ 0.01 (+)	3031.72 $\pm$ 292.85 (+)	3759.83 $\pm$ 610.06 (+)	5358.52 $\pm$ 0.01 (+)	1564.74 $\pm$ 308.05 (+)	5900.45 $\pm$ 122.20 (+)	5982.68 $\pm$ 151.46
		50	5418.52 $\pm$ 0.01 (+)	3900.92 $\pm$ 561.40 (+)	3637.57 $\pm$ 1244.03 (+)	6959.52 $\pm$ 119.20 (+)	1523.54 $\pm$ 390.96 (+)	7661.45 $\pm$ 172.11 (–)	7539.08 $\pm$ 158.81
		80	5508.53 $\pm$ 0.01 (+)	6116.75 $\pm$ 698.19 (+)	2899.20 $\pm$ 1459.92 (+)	8153.21 $\pm$ 206.25 (+)	1557.32 $\pm$ 436.23 (+)	10046.52 $\pm$ 342.33 ($\approx$)	9987.01 $\pm$ 346.65
		100	5568.52 $\pm$ 0.01 (+)	7143.49 $\pm$ 1185.71 (+)	3154.12 $\pm$ 1892.28 (+)	9096.72 $\pm$ 333.84 (+)	1642.84 $\pm$ 450.30 (+)	11740.74 $\pm$ 373.24 ($\approx$)	11734.37 $\pm$ 316.84
Circle	10	5378.52 $\pm$ 0.01 (–)	3026.97 $\pm$ 213.51 (+)	4124.85 $\pm$ 744.27 (+)	5378.52 $\pm$ 0.01 (–)	1664.55 $\pm$ 235.28 (+)	4899.29 $\pm$ 212.97 (+)	5303.87 $\pm$ 23.12	
	30	5598.52 $\pm$ 0.01 (+)	3944.07 $\pm$ 246.23 (+)	4099.16 $\pm$ 773.44 (+)	5598.52 $\pm$ 0.01 (+)	1633.17 $\pm$ 292.18 (+)	5553.40 $\pm$ 284.59 (+)	5993.24 $\pm$ 91.12	
	50	5818.52 $\pm$ 0.01 (+)	5307.06 $\pm$ 493.50 (+)	4824.02 $\pm$ 838.41 (+)	5273.12 $\pm$ 169.71 (+)	1725.65 $\pm$ 313.10 (+)	6934.44 $\pm$ 403.19 ($\approx$)	6853.53 $\pm$ 203.25	
	80	6148.52 $\pm$ 0.01 (+)	7217.47 $\pm$ 483.23 (+)	5898.30 $\pm$ 1450.07 (+)	5613.28 $\pm$ 290.47 (+)	1942.97 $\pm$ 534.96 (+)	8977.22 $\pm$ 578.70 (–)	8090.84 $\pm$ 364.04	
	100	6368.52 $\pm$ 0.01 (+)	8472.43 $\pm$ 592.25 (+)	6646.97 $\pm$ 1903.21 (+)	5872.73 $\pm$ 164.78 (+)	2097.91 $\pm$ 601.03 (+)	10070.07 $\pm$ 728.48 (–)	9153.98 $\pm$ 374.49	
Disturbance	Tanh	10	4603.43 $\pm$ 0.02 (+)	3436.58 $\pm$ 217.20 (+)	4603.44 $\pm$ 0.01 (+)	4603.43 $\pm$ 0.02 (+)	1350.37 $\pm$ 279.60 (+)	5335.56 $\pm$ 64.18 (+)	5718.75 $\pm$ 378.59
		30	4585.38 $\pm$ 0.02 (+)	3695.07 $\pm$ 204.31 (+)	4555.30 $\pm$ 29.44 (+)	4585.38 $\pm$ 0.02 (+)	1500.58 $\pm$ 247.19 (+)	5282.44 $\pm$ 227.15 (+)	5802.61 $\pm$ 431.84
		50	4567.32 $\pm$ 0.02 (+)	4048.70 $\pm$ 298.66 (+)	4296.76 $\pm$ 131.86 (+)	4567.33 $\pm$ 0.02 (+)	1344.44 $\pm$ 196.58 (+)	5193.58 $\pm$ 326.18 (+)	5907.12 $\pm$ 465.54
		80	4540.24 $\pm$ 0.02 (+)	4400.42 $\pm$ 252.77 (+)	3130.82 $\pm$ 894.68 (+)	4540.25 $\pm$ 0.02 (+)	1391.15 $\pm$ 198.17 (+)	4836.97 $\pm$ 153.15 (+)	5670.37 $\pm$ 1125.93
		100	4522.19 $\pm$ 0.03 (+)	4752.82 $\pm$ 214.00 (+)	2073.83 $\pm$ 542.20 (+)	4522.19 $\pm$ 0.03 (+)	1344.13 $\pm$ 238.59 (+)	5075.86 $\pm$ 99.59 (+)	5802.58 $\pm$ 580.49
	Max	10	3950.79 $\pm$ 40.72 (+)	2915.93 $\pm$ 228.05 (+)	4189.18 $\pm$ 306.90 (+)	3951.86 $\pm$ 51.74 (+)	1197.85 $\pm$ 133.49 (+)	4710.30 $\pm$ 33.48 (+)	7362.32 $\pm$ 423.32
		30	4165.75 $\pm$ 76.56 (+)	3034.94 $\pm$ 183.02 (+)	3097.61 $\pm$ 780.14 (+)	4181.54 $\pm$ 55.49 (+)	1283.82 $\pm$ 169.07 (+)	5052.89 $\pm$ 148.06 (+)	7169.14 $\pm$ 462.04
		50	4420.67 $\pm$ 83.09 (+)	4083.73 $\pm$ 424.78 (+)	1265.61 $\pm$ 407.62 (+)	4451.38 $\pm$ 85.27 (+)	1317.51 $\pm$ 222.68 (+)	5556.78 $\pm$ 170.20 (+)	6731.98 $\pm$ 463.16
		80	4797.45 $\pm$ 128.71 (+)	6268.16 $\pm$ 484.68 ($\approx$)	2403.13 $\pm$ 210.49 (+)	4806.64 $\pm$ 127.74 (+)	1484.07 $\pm$ 253.08 (+)	7547.34 $\pm$ 281.93 (–)	6508.85 $\pm$ 529.62
		100	5037.77 $\pm$ 200.04 (+)	7812.62 $\pm$ 577.64 (–)	3337.96 $\pm$ 290.60 (+)	5012.21 $\pm$ 135.17 (+)	1591.21 $\pm$ 258.66 (+)	9161.79 $\pm$ 291.57 (–)	6449.42 $\pm$ 732.43
+ / $\approx$ / –			34 / 2 / 4	34 / 2 / 4	32 / 0 / 8	34 / 2 / 4	40 / 0 / 0	22 / 5 / 13	
Best			5	2	0	6	0	15	18

with strong continuous-circle, continuous-sin, or disturbance-max dependence patterns. This is because PSO+Predictor adopts quadratic functions to simulate the future rewards of solutions, which fits the continuous-circle patterns. Compared with SQL-PSO, CL-PSO performs better on all the instances, showing that the contrastive model can better evaluate the comprehensive reward of solutions in both current and future environments. Compared with CSVC-PSO, CL-PSO performs better on all the instances with a high b value, i.e., $b \geq 30$. This shows that CL-PSO can select high-quality solutions on complex dependence patterns. Compared with FL-SD3, CL-PSO performs better on almost all the instances. This is because the networks are not well-trained under limited data. In addition, the good performance of CL-GA on some instances shows that CLDM can well capture the time-linkage features to improve algorithm performance. CL-PSO performs better than CL-GA since PSO has strong global search ability to provide high-quality data. In general, CL-PSO can well deal with different types and intensities of dependence patterns.

2) *Results on 200-D Instances:* To observe the scalability of CL-PSO, 30 200-D instances are constructed with six different dependence patterns under different b values for test. The results of all the algorithms are reported in Table S.VII of the supplementary material. CL-PSO performs significantly better than PSO, PSO+Predictor, SQL-PSO, CSVC-PSO, FL-SD3, and CL-GA on 28, 29, 25, 28, 30, and 30 out of 30 instances, whereas worse on one, zero, three, two, zero, and zero instances only. In addition, CL-PSO obtains

the best results on 26 instances, followed by SQL-PSO with four instances. This shows that CL-PSO can well predict future rewards on high-dimensional instances. It benefits from the dimensionality reduction operator that discards irrelevant variables.

3) *Computational Time of All Algorithms:* Taking 30 10-D and 200-D instances as examples, the computational time of all the algorithms is reported in Table III. The running time of most algorithms is mainly influenced by problem dimensionality. PSO requires the shortest time with nearly 2.5 s, and SQL-PSO consumes the longest time, longer than 58 s on all the instances. The computational time of CL-PSO is shorter than that of PSO+Predictor, SQL-PSO, FL-SD3, and CL-GA. In addition, the time increment of CL-PSO is nearly 2.5 s from 10-D to 200-D instances, which is smaller than other algorithms with training procedures, i.e., PSO+Predictor, SQL-PSO, and FL-SD3. It is interesting that the time increment involves that of PSO with 2.38 s, indicating that the time increment of CLDM is nearly 0.12 s. This shows that the contrastive learning model in CL-PSO requires a short time on high-dimensional instances due to the dimensionality reduction operator.

4) *Cumulative Fitness Curves of All Algorithms:* To observe the solution quality, taking six 10-D and 200-D instances with different dependency patterns as examples, Figs. 4 and 5 illustrate the cumulative fitness curves of all algorithms along all environments. The accumulative fitness of FL-SD3 increases along time on 10-D instances but decreases drastically on

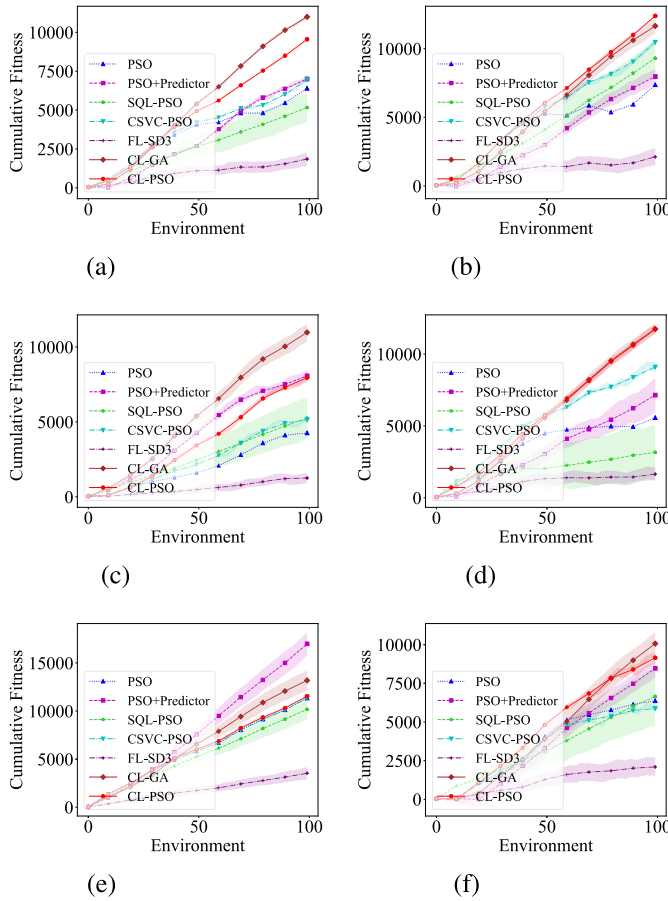


Fig. 4. Cumulative fitness curves of PSO, PSO+Predictor, SQL-PSO, CSVC-PSO, FL-SD3, CL-GA, and CL-PSO for six dependence patterns ($b = 100$) on 10-D instances. (a) Continuous-linear. (b) Discrete-linear. (c) Continuous-sin. (d) Discrete-sin. (e) Continuous-circle. (f) Discrete-circle.

TABLE III

MEAN COMPUTATIONAL TIME (S) OF PSO, PSO+PREDICTOR, SQL-PSO, CSVC-PSO, FL-SD3, CL-GA, AND CL-PSO IN ONE ENVIRONMENT

Mode	Pattern	b	PSO [44]		PSO+Predictor [4]		SQL-PSO [18]		CSVC-PSO [19]		FL-SD3 [45]		CL-GA		CL-PSO	
			10/D	200/D	10/D	200/D	10/D	200/D	10/D	200/D	10/D	200/D	10/D	200/D	10/D	200/D
Linear	Sin	10	0.15	2.70	0.46	13.50	69.33	77.08	0.21	2.79	0.42	6.12	0.39	3.23	0.35	2.93
		30	0.15	2.66	0.47	12.73	61.00	67.22	0.20	2.80	0.41	6.23	0.64	3.42	0.58	2.99
		50	0.15	2.71	0.49	13.37	57.25	70.46	0.20	2.91	0.39	6.29	0.64	3.59	0.58	3.20
		80	0.15	2.70	0.49	12.64	43.41	84.20	0.20	2.89	0.39	6.58	0.62	3.41	0.56	3.21
		100	0.15	2.49	0.48	12.11	39.15	78.50	0.20	2.72	0.39	6.52	0.63	3.41	0.57	3.06
Discrete	Sin	10	0.15	2.39	0.49	14.21	75.50	74.47	0.19	2.53	0.38	6.40	0.28	2.84	0.25	2.66
		30	0.16	2.39	0.47	14.31	79.97	74.48	0.21	2.45	0.38	5.86	0.63	3.22	0.57	2.88
		50	0.16	2.39	0.52	14.29	77.23	63.35	0.20	2.52	0.38	5.70	0.63	2.99	0.57	2.77
		80	0.16	2.39	0.50	15.42	36.76	83.68	0.20	2.56	0.39	5.74	0.64	3.09	0.58	2.81
		100	0.15	2.39	0.52	15.02	34.08	71.58	0.20	2.56	0.39	5.73	0.62	2.93	0.56	2.75
Circle	Sin	10	0.15	2.40	0.49	17.66	63.40	75.14	0.20	2.44	0.38	5.71	0.31	2.80	0.28	2.59
		30	0.15	2.39	0.49	18.56	61.26	76.44	0.20	2.54	0.38	6.12	0.64	3.18	0.58	2.79
		50	0.15	2.39	0.49	17.57	80.64	58.74	0.20	2.42	0.38	6.34	0.65	2.99	0.59	2.71
		80	0.15	2.52	0.55	18.87	60.86	70.36	0.21	2.69	0.38	6.17	0.63	3.22	0.57	2.90
		100	0.16	2.60	0.49	19.56	52.40	74.57	0.21	2.80	0.38	6.25	0.65	3.51	0.59	3.10
Linear	Sin	10	0.16	2.58	0.50	15.81	65.32	77.08	0.20	2.64	0.42	5.72	0.28	3.22	0.25	2.77
		30	0.16	2.58	0.45	9.66	59.06	67.22	0.21	2.81	0.41	5.62	0.66	3.45	0.60	3.22
		50	0.16	2.58	0.46	16.13	70.35	70.46	0.22	2.74	0.40	5.76	0.65	3.44	0.59	3.01
		80	0.16	2.59	0.47	10.19	63.73	84.20	0.21	2.63	0.40	6.81	0.65	3.10	0.59	2.87
		100	0.16	2.59	0.69	14.95	52.17	78.50	0.21	2.65	0.40	6.22	0.66	3.49	0.60	3.15
Continuous	Sin	10	0.16	2.59	0.47	18.49	48.18	74.47	0.20	2.67	0.51	5.99	0.30	3.24	0.27	2.85
		30	0.16	2.48	0.46	12.10	63.73	74.48	0.21	2.54	0.66	5.92	0.64	3.14	0.58	2.86
		50	0.16	2.52	0.46	16.36	59.67	63.35	0.21	2.57	0.80	5.81	0.65	3.23	0.59	2.94
		80	0.16	2.48	0.47	12.72	51.09	83.68	0.21	2.60	0.67	5.86	0.65	3.12	0.59	2.80
		100	0.16	2.47	0.48	17.53	48.81	71.58	0.21	2.58	0.58	5.80	0.64	3.11	0.58	2.83
Circle	Sin	10	0.16	2.47	0.45	15.81	59.24	75.14	0.20	2.57	0.58	5.77	0.28	3.12	0.25	2.71
		30	0.16	2.48	0.47	12.36	48.60	76.44	0.20	2.52	0.58	5.83	0.29	2.84	0.26	2.65
		50	0.16	2.52	0.60	15.33	57.94	58.74	0.20	2.58	0.58	5.78	0.54	3.33	0.49	2.99
		80	0.16	2.47	0.94	24.07	61.19	70.36	0.21	2.66	0.59	5.84	0.57	3.11	0.52	2.81
		100	0.16	2.47	0.97	25.09	56.81	74.57	0.21	2.62	0.59	5.78	0.62	3.28	0.56	2.90

200-D instances. Similar performance degeneration can be seen on PSO+Predictor. In contrast, SQL-PSO, CSVC-PSO, and CL-PSO maintain performance even in high-dimensional spaces due to the adoption of dimensionality reduction operators. Specially, SQL-PSO performs the best on one 200-D instance with continuous-circle patterns. CL-PSO obtains the best results on four 200-D instances, surpassing CL-GA due to the search efficiency of PSO.

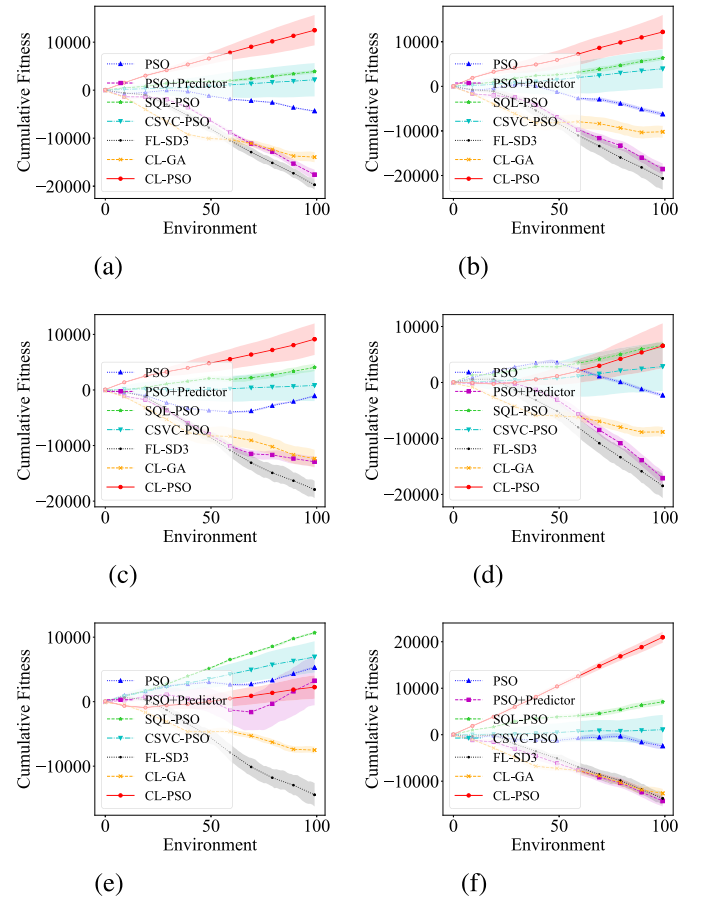


Fig. 5. Cumulative fitness curves of PSO, PSO+Predictor, SQL-PSO, CSVC-PSO, FL-SD3, CL-GA, and CL-PSO for six dependence patterns ($b = 100$) on 200-D instances. (a) Continuous-linear. (b) Discrete-linear. (c) Continuous-sin. (d) Discrete-sin. (e) Continuous-circle. (f) Discrete-circle.

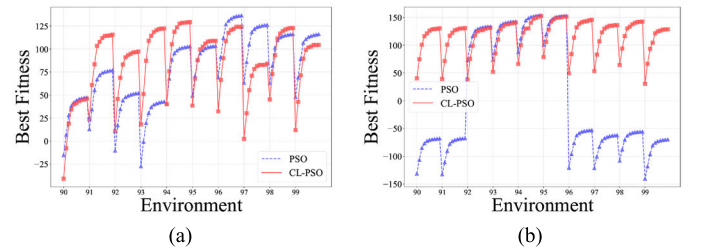


Fig. 6. Convergence curves of PSO and CL-PSO on 10-D instances with (a) continuous-linear and (b) discrete-linear patterns under $b = 100$.

5) *Search Behaviors of CL-PSO*: To further observe the convergence behavior of CL-PSO, taking the 90th–99th environments of two 10-D instances with continuous-linear and discrete-linear patterns under $b = 100$ as examples, Fig. 6 illustrates the convergence curve of PSO and CL-PSO. PSO and CL-PSO present similar search behavior. Compared with PSO, CL-PSO can obtain better solutions in most environments. This shows that CL-PSO can select decision solutions with higher future rewards in most environments.

To observe the selected decision solutions, the fitness of the final swarm and the decision solution in the first and second environments per five environments are plotted in Fig. 7. In the early stage, CL-PSO performs similar to PSO since the contrastive learning model is not adopted. After fifth

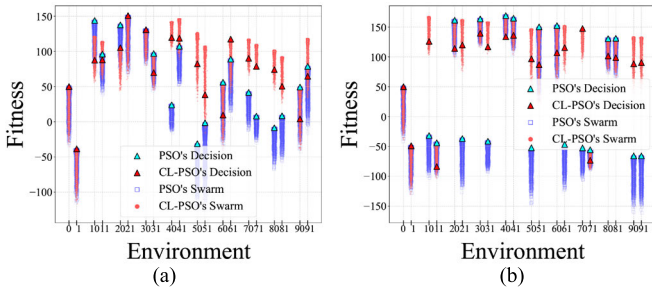


Fig. 7. Distributions of the fitness of the final swarm and selected decision solution in CL-PSO and PSO every five environments on 10-D instances with (a) continuous-linear and (b) discrete-linear patterns under $b = 100$.

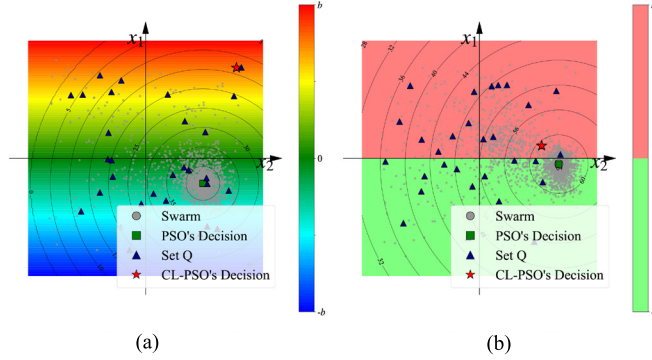


Fig. 8. Distributions of swarms (all solutions) found by CL-PSO and PSO in the 49th environment of the 10-D instances with (a) continuous-linear and (b) discrete-linear patterns under $b = 100$. The horizontal and vertical axes represent the first and second dimensions of search space, respectively. Different colors represent γ values of solutions. The black circles represent the fitness contours in the current environment.

environment, CL-PSO tends to select solutions with lower fitness from the swarm to obtain higher future rewards. For example, in the 40th and 41th environments of continuous-linear instance, CL-PSO finds better solutions than PSO. In this way, CL-PSO can obtain better accumulative fitness along all environments. The advantage of CL-PSO is more obvious as the environment index increases. This is because the growing dataset helps increase prediction accuracy of the contrastive learning model.

To observe decision-making process, Fig. 8 illustrates the distribution of the candidate solutions (swarms) found by PSO and CL-PSO in the search space of the 49th environment. The swarm of PSO converges toward the optimum of the current environment. The selected decision solution of PSO has high current fitness but low future reward (b value in the 50th environment), resulting in poor accumulative value. In contrast, CL-PSO first clusters the candidate solutions found by the swarm into multiple subsets (blue triangles), which involve solutions with high current fitness or ones with high b values. CL-PSO selects a decision solution (red pentagram) through the pairwise comparisons by the contrastive learning model. This achieves the favorable tradeoff between immediate fitness in the current environment and predicted future rewards.

E. Comparison to Other Algorithms Under Different Changing Patterns of Optima

To test the performance of CL-PSO on instances with different changing patterns of optima, λ is set in a range

of $[0, 1]$ with a step size of 0.2 for test. When $\lambda = 1$, the environmental change follows a stable pattern; when $\lambda = 0$, the environment changes randomly. Instances with continuous-linear, continuous-circle, discrete-linear, and discrete-circle dependence patterns are constructed for test under b of 10, 50, and 100. That is, a total of 72 instances are tested. The results of all the algorithms on instances with continuous dependence patterns are reported in Table S.VIII, and the results on discrete dependence patterns are listed in Table S.IX of the supplementary material.

As shown in Table S.VIII, CL-PSO performs significantly better than PSO, PSO+Predictor, SQL-PSO, and CSVC-PSO on 30, 24, 36, and 30 out of 36 instances, whereas worse on one, 12, zero, and one instances only. In addition, CL-PSO obtains the best results on 18 instances, followed by PSO+Predictor and CSVC-PSO with 12 and five instances. Particularly, PSO+Predictor is better on instances with continuous-circle patterns due to the adoption of the quadratic predictor. However, PSO+Predictor performs poorly on other types of dependence patterns. The better performance of CL-PSO on most instances shows that CL-PSO can well deal with different changing patterns of optimal solutions. From Table S.IX, CL-PSO performs significantly better than PSO, PSO+Predictor, SQL-PSO, and CSVC-PSO on 24, 36, 36, and 24 out of 36 instances, whereas worse on six, zero, zero, and six instances only. CL-PSO performs the best on 25 instances, followed by PSO and CSVC-PSO with seven and six instances. In particular, CL-PSO is slightly worse than CSVC-PSO on a few instances with $b \leq 50$ but outperforms CSVC-PSO in all the instances with $b = 100$. This shows that the proposed method can well solve instances with strong time linkages. In general, CL-PSO performs better on most instances with different changing patterns of optima.

F. Comparison to Other Algorithms on Multiobjective Optimization

To verify the performance of CLDM on dynamic multiobjective optimization, we integrate CLDM with NSGA-II [48], resulting in CLDM-NSGA-II. DF1-DF5 in the CEC2018 dynamic test suite [49] are taken for test. To introduce time-linkage features into DF1-DF5, in each environment, one solution with the minimum sum of all the objectives is selected from solution sets as decision. Denote the first variable value in the decision solution as x_1 . The objective functions in the subsequent environment are added with a value of $\text{sign}(x_1)b$. Particularly, b is set to 0.5, 1.0, and 2.0 for each problem, yielding a total of 15 instances. The cumulative hypervolume (HV) over 100 environments is calculated as performance measure. Two typical multiobjective optimization algorithms, i.e., NSGA-II and MOEA/D [50], and two recent ones, i.e., RL-REVA [51] and TPDCB [52], are taken for comparison.

The mean accumulative HV results over 30 runs are presented in Table S.X of the supplementary material. CLDM-NSGA-II demonstrates significantly better performance compared with NSGA-II, MOEA/D, RL-REVA, and TPDCB on 11, 12, 12, and 12 out of the 15 instances, whereas worse on only two, two, two, and two instances, respectively. Notably, CLDM-NSGA-II achieves the best HV results on

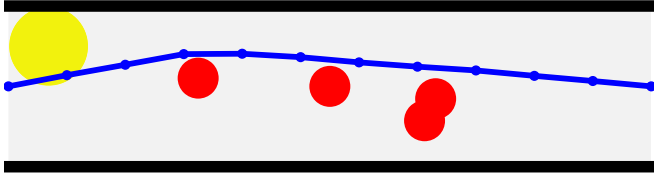


Fig. 9. Example of mobile robot path planning. The gray area represents a road segment. The yellow circle represents a puddle and red circles represent obstacles. The blue line is a feasible path of the robot.

12 instances. On instances with weak time linkages ($b = 0.5$), CLDM-NSGA-II performs slightly worse than other algorithms on DF1 and DF3. On instances with strong time-linkage intensity ($b = 1.0$ and $b = 2.0$), CLDM-NSGA-II outperforms all the competing algorithms. This shows the efficiency of CLDM in learning the dependencies between decisions and subsequent environmental states, thereby achieving superior long-term performance on dynamic multiobjective optimization.

G. Application to Mobile Robot Path Planning

Mobile robots are usually adopted to execute tasks under dangerous environments. The path planning of robots is a critical problem to generate optimal and collision-avoiding paths for robots. The path planning problem is a dynamic time-linkage optimization problem [53] since the traffic changes with time and is impacted by the path of robots. Therefore, it is appealing to apply CL-PSO to path planning.

An example of the path planning problem is illustrated in Fig. 9, in which the road segment is covered with coal ash due to traffic accidents. There are several obstacles (red circles) and a puddle (yellow circle). Once a robot passes through the road segment, the coal ash will be carried into air to cause low visibility, which affects the travel of robots for 80 s until sedimentation. Specially, if the robot travels over the puddle, then its wheels get wet and will not bring coal ash to air. The robots can travel with a normal speed of 1 m/s but 0.1 m/s for safe under low visibility.

The path of a robot can be defined by p points which are fit using B-Spline. For the i th sample point, the x -axis coordinate value is implicitly set relative to i/p , while the y -axis coordinate values of the sample points need to be defined by optimization algorithms to form a complete path. In this way, the path planning problem is converted into a p -dimensional optimization problem. In the experiment, 100 robots are required to travel the road segment sequentially. To ensure safety, each robot is allowed to enter the road segment only once the preceding robot has reached the destination. The path planning of one robot represents a time window, and the sequential traversal of 100 robots constitutes 100 different environments due to the visibility changes. The optimization objective is to minimize the total traveling time of all robots.

Two state-of-the-art algorithms for robot path planning are adopted for comparison, i.e., dynamic parameter-A* (DP-A*) [54] and information-assisted double dueling deep Q -network (IA-D3QN) [55]. Particularly, DP-A* uses Q -learning to modify evaluation functions and records the characteristic of changes. IA-D3QN adopts an information compression

TABLE IV
RESULTS OF DP-A*, IA-D3QN, PSO, PSO+PREDICTOR, SQL-PSO, CSVC-PSO, AND CL-PSO ON THE MOBILE ROBOT PATH PLANNING PROBLEM

Algorithm	Travel Time (seconds)
DP-A* [54]	6049.25 $\pm$ 517.90 (+)
IA-D3QN [55]	3911.02 $\pm$ 319.51 (+)
PSO [44]	6078.66 $\pm$ 303.48 (+)
PSO+Predictor [4]	3976.90 $\pm$ 422.61 (+)
SQL-PSO [18]	3858.99 $\pm$ 705.08 (+)
CSVC-PSO [19]	3857.59 $\pm$ 191.03 (+)
CL-PSO	3475.15 $\pm$ 479.03

scheme to train the mutual information between network layers to improve the generalization ability of reinforcement learning. Each algorithm runs 30 times. The mean values and standard deviations of the traveling time over all runs are reported. As shown in Table IV, DP-A*, IA-D3QN, PSO, PSO+Predictor, SQL-PSO, and CSVC-PSO require 6049.25, 3911.02, 6078.66, 3976.90, 3858.99, and 3857.59 s, respectively. The performance of IA-D3QN is comparable to PSO, indicating that it primarily optimizes current robot's immediate paths but ignores the inherent time-linkage property. In contrast, CL-PSO requires the shortest time of 3475.15 s, which is significantly shorter than other algorithms. The proposed method can find better paths for robots in the long term by effectively dealing with the time-linkage feature.

V. CONCLUSION

To address the future reward evaluation and solution selection issues in dynamic time-linkage optimization, this article proposes CL-PSO. In CL-PSO, a CD strategy is adopted to check the intensity of time linkages. Based on the time-linkage intensity, a contrastive model is trained to learn the relationship between solutions in terms of the combination of current fitness and future rewards. Candidate solutions are filtered through clustering and ranked using the contrastive model to select decision solutions. The experimental results show that CL-PSO outperforms state-of-the-art algorithms on most instances with different types and intensities of dependence patterns. In addition, each component of CL-PSO helps improve algorithm performance. Particularly, the CD can evaluate the time-linkage intensity to determine whether to introduce future rewards for decision making. The contrastive model can predict the relationships between solutions in terms of current fitness and future rewards to assist decision making. Quick decision helps reduce the computational time.

Several avenues are open for further investigations. First, since the contrastive models are developed for problems with strong time linkages, new methods could be developed to deal with instances with weak time linkages. Second, this article focuses on eight dependence patterns unchanged over time. New methods could be proposed to solve more complex dependence patterns that change with time.

REFERENCES

- [1] B. Werth, E. Pitzer, J. Karder, S. Wagner, and M. Affenzeller, "Dynamic vehicle routing with time-linkage: From problem states to algorithm performance," in *Proc. Int. Conf. Comput. Aided Syst. Theory*, 2022, pp. 69–77.

- [2] S. Hahn-Goldberg, M. W. Carter, J. C. Beck, M. Trudeau, P. Sousa, and K. Beattie, "Dynamic optimization of chemotherapy outpatient scheduling with uncertainty," *Health Care Manage. Sci.*, vol. 17, no. 4, pp. 379–392, Dec. 2014.
- [3] J. Chai, W. Chen, Y. Zhu, Z.-X. Yao, and D. Zhao, "A hierarchical deep reinforcement learning framework for 6-DOF UCAV air-to-air combat," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 53, no. 9, pp. 5417–5429, Sep. 2023.
- [4] T. T. Nguyen, Z. Yang, and S. Bonsall, "Dynamic time-linkage problems—The challenges," in *Proc. IEEE RIVF Int. Conf. Comput. Commun. Technol., Res., Innov., Vis. Future*, Feb. 2012, pp. 1–6.
- [5] D. Yazdani, R. Cheng, D. Yazdani, J. Branke, Y. Jin, and X. Yao, "A survey of evolutionary continuous dynamic optimization over two decades—Part B," *IEEE Trans. Evol. Comput.*, vol. 25, no. 4, pp. 630–650, Aug. 2021.
- [6] Y. Hu, J. Zheng, S. Jiang, S. Yang, J. Zou, and R. Wang, "A Mahalanobis distance-based approach for dynamic multiobjective optimization with stochastic changes," *IEEE Trans. Evol. Comput.*, vol. 28, no. 1, pp. 238–251, Feb. 2024.
- [7] L. Feng, W. Zhou, W. Liu, Y.-S. Ong, and K. C. Tan, "Solving dynamic multiobjective problem via autoencoding evolutionary search," *IEEE Trans. Cybern.*, vol. 52, no. 5, pp. 2649–2662, May 2022.
- [8] X.-F. Liu et al., "Neural network-based information transfer for dynamic optimization," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 5, pp. 1557–1570, May 2020.
- [9] X. Zhang, G. Yu, Y. Jin, and F. Qian, "Elitism-based transfer learning and diversity maintenance for dynamic multi-objective optimization," *Inf. Sci.*, vol. 636, Jul. 2023, Art. no. 118927.
- [10] F. Zou and G. G. Yen, "Dynamic multiobjective optimization with varying number of objectives assisted by dynamic principal component analysis," *Inf. Sci.*, vol. 665, Apr. 2024, Art. no. 120398.
- [11] Q. Lin, Y. Ye, L. Ma, M. Jiang, and K. C. Tan, "Dynamic multiobjective evolutionary optimization via knowledge transfer and maintenance," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 54, no. 2, pp. 936–949, Feb. 2024.
- [12] W. Zheng, H. Chen, and X. Yao, "Analysis of evolutionary algorithms on fitness function with time-linkage property," *IEEE Trans. Evol. Comput.*, vol. 25, no. 4, pp. 696–709, Aug. 2021.
- [13] T. Yang and Y. Zhou, "Analysis of multiobjective evolutionary algorithms on fitness function with time-linkage property," *IEEE Trans. Evol. Comput.*, vol. 28, no. 3, pp. 837–843, Jun. 2024.
- [14] P. A. N. Bosman, "Learning, anticipation and time-deception in evolutionary online dynamic optimization," in *Proc. 7th Annu. workshop Genetic Evol. Comput.*, Jun. 2005, pp. 39–47.
- [15] C. Bu, W. Luo, T. Zhu, and L. Yue, "Solving online dynamic time-linkage problems under unreliable prediction," *Appl. Soft Comput.*, vol. 56, pp. 702–716, Jul. 2017.
- [16] W. Zheng, Q. Zhang, H. Chen, and X. Yao, "When non-elitism meets time-linkage problems," in *Proc. Genetic Evol. Comput. Conf.*, Jun. 2021, pp. 741–749.
- [17] H. Fu, P. R. Lewis, B. Sendhoff, K. Tang, and X. Yao, "What are dynamic optimization problems?" in *Proc. IEEE Congr. Evol. Comput. (CEC)*, Jul. 2014, pp. 1550–1557.
- [18] T. Zhang, H. Wang, B. Yuan, Y. Jin, and X. Yao, "Surrogate-assisted evolutionary Q-learning for black-box dynamic time-linkage optimization problems," *IEEE Trans. Evol. Comput.*, vol. 27, no. 5, pp. 1162–1176, Oct. 2023.
- [19] M. Gao, X.-F. Liu, Z. Zhan, and J. Zhang, "A clustering-based support vector classifier for dynamic time-linkage optimization," in *Proc. IEEE Symp. Ser. Comput. Intell.*, Feb. 2023, pp. 953–958.
- [20] M. Haris, D. M. S. Bhatti, and H. Nam, "A fast-convergent hyperbolic tangent PSO algorithm for UAVs path planning," *IEEE Open J. Veh. Technol.*, vol. 5, pp. 681–694, 2024.
- [21] X.-F. Liu, X.-X. Xu, Z.-H. Zhan, Y. Fang, and J. Zhang, "Interaction-based prediction for dynamic multiobjective optimization," *IEEE Trans. Evol. Comput.*, vol. 27, no. 6, pp. 1881–1895, Jun. 2023.
- [22] C. Li, S. Yang, and M. Yang, "An adaptive multi-swarm optimizer for dynamic optimization problems," *Evol. Comput.*, vol. 22, no. 4, pp. 559–594, Dec. 2014.
- [23] A. M. Turky and S. Abdullah, "A multi-population harmony search algorithm with external archive for dynamic optimization problems," *Inf. Sci.*, vol. 272, pp. 84–95, Jul. 2014.
- [24] S. Dennis and A. Engelbrecht, "Dynamic multi-swarm fractional-best particle swarm optimization for dynamic multi-modal optimization," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, Dec. 2020, pp. 1549–1556.
- [25] L. Liu, S. Yang, and D. Wang, "Particle swarm optimization with composite particles in dynamic environments," *IEEE Trans. Syst., Man, Cybern., B, Cybern.*, vol. 40, no. 6, pp. 1634–1648, Dec. 2010.
- [26] M. C. Du Plessis and A. P. Engelbrecht, "Differential evolution for dynamic environments with unknown numbers of optima," *J. Global Optim.*, vol. 55, no. 1, pp. 73–99, Jan. 2013.
- [27] C. Rossi, M. Abderrahim, and J. C. Díaz, "Tracking moving optima using Kalman-based predictions," *Evol. Comput.*, vol. 16, no. 1, pp. 1–30, Mar. 2008.
- [28] A. Simões and E. Costa, "Evolutionary algorithms for dynamic environments: Prediction using linear regression and Markov chains," in *Proc. 10th Int. Conf. Parallel Problem Solving Nature*, 2008, pp. 306–315.
- [29] A. Ahrari, S. Elsayed, R. Sarker, D. Essam, and C. A. C. Coello, "Adaptive multilevel prediction method for dynamic multimodal optimization," *IEEE Trans. Evol. Comput.*, vol. 25, no. 3, pp. 463–477, Jun. 2021.
- [30] J. L. Salmeron and W. Froelich, "Dynamic optimization of fuzzy cognitive maps for time series forecasting," *Knowl.-Based Syst.*, vol. 105, pp. 29–37, Aug. 2016.
- [31] A. Zhou, Y. Jin, and Q. Zhang, "A population prediction strategy for evolutionary dynamic multiobjective optimization," *IEEE Trans. Cybern.*, vol. 44, no. 1, pp. 40–53, Jan. 2014.
- [32] X.-F. Liu, Z.-H. Zhan, and J. Zhang, "Transfer-based particle swarm optimization for large-scale dynamic optimization with changing variable interactions," *IEEE Trans. Evol. Comput.*, vol. 28, no. 6, pp. 1633–1643, Dec. 2024.
- [33] M. Jiang, Z. Huang, L. Qiu, W. Huang, and G. G. Yen, "Transfer learning-based dynamic multiobjective optimization algorithms," *IEEE Trans. Evol. Comput.*, vol. 22, no. 4, pp. 501–514, Aug. 2018.
- [34] H. Zhang, J. Ding, M. Jiang, K. C. Tan, and T. Chai, "Inverse Gaussian process modeling for evolutionary dynamic multiobjective optimization," *IEEE Trans. Cybern.*, vol. 52, no. 10, pp. 11240–11253, Oct. 2022.
- [35] D. Xu, M. Jiang, W. Hu, S. Li, R. Pan, and G. G. Yen, "An online prediction approach based on incremental support vector machine for dynamic multiobjective optimization," *IEEE Trans. Evol. Comput.*, vol. 26, no. 4, pp. 690–703, Aug. 2022.
- [36] Y. Ye, L. Li, Q. Lin, K.-C. Wong, J. Li, and Z. Ming, "Knowledge guided Bayesian classification for dynamic multi-objective optimization," *Knowl.-Based Syst.*, vol. 250, Aug. 2022, Art. no. 109173.
- [37] M. Jiang, Z. Wang, L. Qiu, S. Guo, X. Gao, and K. C. Tan, "A fast dynamic evolutionary multiobjective algorithm via manifold transfer learning," *IEEE Trans. Cybern.*, vol. 51, no. 7, pp. 3417–3428, Jul. 2021.
- [38] G. Chen, Y. Guo, M. Huang, D. Gong, and Z. Yu, "A domain adaptation learning strategy for dynamic multiobjective optimization," *Inf. Sci.*, vol. 606, pp. 328–349, Aug. 2022.
- [39] Q. Zhao, B. Yan, Y. Shi, and M. Middendorf, "Evolutionary dynamic multiobjective optimization via learning from historical search process," *IEEE Trans. Cybern.*, vol. 52, no. 7, pp. 6119–6130, Jul. 2022.
- [40] D. N. Reshef et al., "Detecting novel associations in large data sets," *Science*, vol. 334, no. 6062, pp. 1518–1524, Dec. 2011.
- [41] Q. Que and M. Belkin, "Back to the future: Radial basis function network revisited," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 8, pp. 1856–1867, Aug. 2020.
- [42] H.-G. Huang and Y.-J. Gong, "Contrastive learning: An alternative surrogate for offline data-driven evolutionary computation," *IEEE Trans. Evol. Comput.*, vol. 27, no. 2, pp. 370–384, Apr. 2023.
- [43] B. Fornberg, E. Larsson, and N. Flyer, "Stable computations with Gaussian radial basis functions," *SIAM J. Scientific Comput.*, vol. 33, no. 2, pp. 869–892, Jan. 2011.
- [44] A. G. Gad, "Particle swarm optimization algorithm and its applications: A systematic review," *Arch. Comput. Methods Eng.*, vol. 29, no. 5, pp. 2531–2561, Aug. 2022.
- [45] S. Zhou, L. Feng, M. Mei, and M. Yao, "Dynamic resource management for federated edge learning with imperfect CSI: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 11, no. 18, pp. 30400–30412, Sep. 2024.
- [46] S. Forrest, "Genetic algorithms," *ACM Comput. Surv.*, vol. 28, no. 1, pp. 77–80, 1996.

- [47] J. H. Zar, "Significance testing of the Spearman rank correlation coefficient," *J. Amer. Stat. Assoc.*, vol. 67, no. 339, pp. 578–580, Sep. 1972.
- [48] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.
- [49] S. Jiang, S. Yang, X. Yao, K. C. Tan, M. Kaiser, and N. Krasnogor, "Benchmark functions for the CEC'2018 competition on dynamic multiobjective optimization," Newcastle University, Newcastle Upon Tyne, U.K., Tech. Rep., 2018.
- [50] Q. Zhang and H. Li, "MOEA/D: A multiobjective evolutionary algorithm based on decomposition," *IEEE Trans. Evol. Comput.*, vol. 11, no. 6, pp. 712–731, Dec. 2007.
- [51] P. Liang, Y. Chen, Y. Sun, Y. Huang, and W. Li, "An information entropy-driven evolutionary algorithm based on reinforcement learning for many-objective optimization," *Expert Syst. Appl.*, vol. 238, Mar. 2024, Art. no. 122164.
- [52] C. Zhu, Y. Yang, X. Kong, and Y. Yang, "A dual dynamic constraint boundary based constrained multi-objective evolutionary algorithm for small feasible regions," *Expert Syst. Appl.*, vol. 275, May 2025, Art. no. 127008.
- [53] L. Zhang, Y. Zhang, and Y. Li, "Mobile robot path planning based on improved localized particle swarm optimization," *IEEE Sensors J.*, vol. 21, no. 5, pp. 6962–6972, Mar. 2021.
- [54] X. Gan, Z. Huo, and W. Li, "DP—A: For path planing of UGV and contactless delivery," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 1, pp. 907–919, Jan. 2024.
- [55] M. Xi, J. Yang, J. Wen, Z. Li, W. Lu, and X. Gao, "An information-assisted deep reinforcement learning path planning scheme for dynamic and unknown underwater environment," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 1, pp. 842–853, Jan. 2025.



Xiao-Fang Liu (Senior Member, IEEE) received the B.S. and Ph.D. degrees in computer science from Sun Yat-sen University, Guangzhou, China, in 2015 and 2020, respectively.

She is currently an Associate Professor with College of Artificial Intelligence, Nankai University, Tianjin, China. Her current research interests include artificial intelligence, evolutionary computation, swarm intelligence, and their applications in design and optimization such as cloud computing resources scheduling and multirobot systems.



Meng Gao received the B.S. and M.S. degrees in intelligence science and technology from Nankai University, Tianjin, China, in 2022 and 2025, respectively.

His current research interests include artificial intelligence, evolutionary computation, swarm intelligence, and their applications in design and optimization such as multirobot systems.



Yongchun Fang (Senior Member, IEEE) received the B.S. degree in electrical engineering and the M.S. degree in control theory and applications from Zhejiang University, Hangzhou, China, in 1996 and 1999, respectively, and the Ph.D. degree in electrical engineering from Clemson University, Clemson, SC, USA, in 2002.

He is currently a Professor with the Institute of Robotics and Automatic Information System (IRAI), Nankai University, Tianjin, China. His research interests include nonlinear control, visual servoing, AFM-based nano-systems, and control of underactuated systems.



Zhi-Hui Zhan (Fellow, IEEE) received the bachelor's and Ph.D. degrees in computer science from the Sun Yat-sen University, Guangzhou, China, in 2007 and 2013, respectively.

He is currently a Changjiang Scholar Young Professor and a Gifted Professor with the College of Artificial Intelligence, Nankai University, Tianjin, China. His current research interests include evolutionary computation, swarm intelligence, and their applications in real-world problems.

Prof. Zhan was a recipient of the IEEE Computational Intelligence Society (CIS) Outstanding Early Career Award in 2021, the Outstanding Youth Science Foundation from the National Natural Science Foundation of China (NSFC) in 2018, and the Wu Wen-Jun Artificial Intelligence Excellent Youth from Chinese Association for Artificial Intelligence in 2017. He is listed as the Highly Cited Researcher by Clarivate Analytics, the World's Top 2% Scientist for both Career-Long Impact and Year Impact on Artificial Intelligence, and the Elsevier Highly Cited Chinese Researcher in Computer Science. He is an Associate Editor of IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION, IEEE TRANSACTIONS ON EMERGING TOPICS IN COMPUTATIONAL INTELLIGENCE, IEEE TRANSACTIONS ON SYSTEMS, MAN AND CYBERNETICS: SYSTEMS, and IEEE TRANSACTIONS ON ARTIFICIAL INTELLIGENCE.



Jun Zhang (Fellow, IEEE) received the Ph.D. degree in electrical engineering from the City University of Hong Kong, Hong Kong, in 2002.

His primary research interests lie in the field of computational intelligence (CI), with a strong focus on evolutionary computation (EC) and its wide-ranging applications. He has authored more than 600 peer-reviewed publications, including more than 230 papers in IEEE TRANSACTIONS. He has been recognized as a Clarivate Highly Cited Researcher, ranking within the top 1% worldwide in the field of computer science.

Dr. Zhang received the Outstanding Young Scientist Fund by the National Natural Science Foundation of China (NSFC) in 2011. He currently serves as an Associate Editor for IEEE TRANSACTIONS ON ARTIFICIAL INTELLIGENCE and IEEE TRANSACTIONS ON CYBERNETICS.